



COMSATS University Islamabad, Sahiwal
Off COMSATS Road, Sahiwal

SAFAR

(Version 1.0)

A project presented to
COMSATS University, Islamabad

In partial fulfillment
of the requirement for the degree of

Bachelor of Science in Computer Science (2021-2025)

By

Fatima Naseer

CIIT/FA21-BCS-015/SWL

Khushbakht Fatima

CIIT/FA21-BCS-044/SWL

DECLARATION

We hereby declare that this software, neither whole nor as a part, has been copied out from any source. It is further declared that we have developed this software and accompanied the report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Fatima Naseer

Khushbakht Fatima

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CS) “Safar” was developed by **Fatima Naseer (CIIT/FA21-BCS-015/SWL)** and **Khushbakht Fatima (CIIT/FA21-BCS-044/SWL)** under the supervision of “**Ms. Tahreem Saeed**” and co supervisor “**Dr. Zafar Iqbal Roy**” and that in (their/his/her) opinion; it is fully adequate, in scope and quality for the degree of Bachelor of Science in Computer Sciences.

Supervisor

External Examiner

Head of Department
(Department of Computer Science)

Executive Summary

In a country where inter-city travel is a common necessity, the current bus travel system faces numerous challenges including lack of integration, inefficient ticketing processes, limited user personalization, and communication gaps between passengers and service providers. Travelers often face difficulties in accessing reliable schedules, comparing fares, or booking seats that meet their preferences, especially in terms of comfort and privacy.

To address these issues, a comprehensive and user-friendly travel application has been developed. This mobile-based platform aims to simplify and modernize the way people plan and experience inter-city bus travel. It brings together multiple bus services under one digital roof, enabling passengers to book tickets, make online payments, apply for refund, complain, make suggestions, and choose seats based on gender preferences, thus ensuring convenience and cultural sensitivity.

The travel app also offers access to real-time bus schedules, detailed route information, and fare comparison tools, allowing users to make informed decisions quickly and efficiently. A feedback system is integrated to gather user reviews, which helps improve service quality over time.

On the operator side, the app provides agencies with the ability to create and manage their profiles, set schedules, upload routes, and monitor bookings, giving them greater control and reducing dependency on traditional manual processes. An admin page is built to verify the status of the newly registered companies. A company will be given access to their account only if they satisfy the quality services criteria.

By bridging the gap between passengers and bus operators, the app delivers a modern solution that enhances the overall travel experience, ensures operational transparency, and promotes safe, efficient, and reliable bus transportation across Pakistan.

.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task. His blessings, guidance, and wisdom have been the driving force behind our efforts and the successful completion of this project. We are truly grateful for the strength, patience, and determination granted to us throughout this journey.

We are greatly indebted to our project supervisor **Ms. Tahreem Saeed** and our co-supervisor **Dr. Zafar Iqbal Roy**. Without their personal supervision, advice, and valuable guidance, completion of this project would have been doubtful. Their dedication, timely feedback, and insightful suggestions provided us with the direction we needed. We are deeply indebted to them for their encouragement and continual help during this work. Their constant motivation and expertise were instrumental in shaping our project to its current form.

We are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty and hard work. Their unconditional love, prayers, and support provided the emotional and moral strength we needed to stay focused and committed. Without their sacrifices and belief in us, this achievement would not have been possible.

Fatima Naseer

Khushbakht Fatima

Table of Contents

1. Introduction.....	1
1.1 Brief	1
1.2 Relevance to Course Modules	3
1.3 Project Background.....	5
1.4 Literature Review.....	6
1.4.1 Existing System Analysis	6
1.5 Analysis from literature review	7
1.6 Methodology and Software Development Life Cycle	8
1.6.1 Methodology and SDLC Selected	8
1.6.2 Rationale behind selecting methodology	9
2. Problem Definition.....	11
2.1 Problem Statement.....	11
2.2 Problem Solution	11
2.3 Objectives of the Proposed System.....	12
2.4 Scope.....	13
2.5 Modules.....	14
2.5.1 Module 1: User Profiling	14
2.5.2 Module 2: Ticket Booking.....	15
2.5.3 Module 3: Admin Panel (Company Admin).....	15
2.5.4 Module 4: Admin Panel (Main Admin).....	16
2.5.5 Module 5: Additional Features	16
2.6 Deliverable and Development Requirements	17
2.6.1 Deliverables	17
2.6.2 Development Requirements.....	17

3. Requirement Analysis.....	19
3.1 Requirement Identifying Technique	19
3.1.1 Use Case Diagram.....	19
3.1.2 Tabular Use Cases.....	22
3.2 Functional Requirements	27
3.2.1 Ticket Booking Module	27
3.2.2 Online Payment Module	27
3.2.3 Bus Schedule and Fare Comparison Module.....	27
3.2.4 Feedback and History	28
3.3 Non-Functional Requirements	28
3.3.1 Usability.....	28
3.3.2 Performance	28
3.3.3 Security	29
3.3.4 Portability.....	29
3.3.5 Reliability.....	29
3.3.6 Maintainability.....	29
3.3.7 Scalability	30
4. Design and Architecture.....	32
4.1 System Architecture.....	32
4.2 Design Models	32
4.2.1 Activity Diagram	32
4.3 Data Definition Diagram.....	39
4.4 Sequence Diagram	40
4.5 State Transition Diagram	42
4.6 Data Design.....	44

4.6.1	Data Dictionary	44
5.	Implementation	52
5.1	Algorithm	52
5.1.1	User Profiling.....	52
5.1.2	Company Admin.....	53
5.1.3	User Side	55
5.2	External APIs	58
5.3	User Interface.....	59
6.	Testing and Evaluation.....	71
6.1	Unit Testing	71
6.2.	Functional Testing	79
6.3.	Integration Testing	82
7.	Conclusion and Future Work	87
7.1	Conclusion	87
7.2	Future Work	87
References		89

List of Figures

Figure 3.1: Common Use Case Diagram	19
Figure 3.2: Admin Use Case Diagram.....	20
Figure 3.3: User Use Case Diagram	20
Figure 3.4: Company Admin Use Case Diagram	21
Figure 4.1: MVC System Architecture Diagram	32
Figure 4.2: Activity Diagram of Authentication.....	34
Figure 4.3: Activity Diagram of Admin	35
Figure 4.4: Activity Diagram of Company Admin.....	36
Figure 4.5: Activity Diagram of User	38
Figure 4.6: Class Diagram of Safar.....	39
Figure 4.7: Sequence Diagram of Company Admin.....	40
Figure 4.8: Sequence Diagram of User	41
Figure 4.9: Sequence Diagram of Admin	42
Figure 4. 10: State Diagram of Payment Verification	43
Figure 4. 11: State Diagram of Refund Process.....	44
Figure 5.1: Login UI	59
Figure 5.2: Signup UI	59
Figure 5.3: Companies Verification UI	60
Figure 5.4: Users Verification UI	60
Figure 5.5: Homepage UI	60
Figure 5.6: Filter UI.....	60
Figure 5.7: Available Tickets UI	61
Figure 5.8: Choose Yourself UI.....	61
Figure 5.9: Bus Seat UI.....	61
Figure 5.10: Ticket Detail UI.....	61
Figure 5.11: JazzCash UI.....	62
Figure 5.12: EasyPaisa UI.....	62
Figure 5.13: Booked Ticket UI	62
Figure 5.14: Booking History UI.....	62
Figure 5.15: Notifications UI.....	63

Figure 5.16: Drawer UI.....	63
Figure 5.17: User Profile UI	63
Figure 5.18: Edit Profile UI	63
Figure 5.19: Contact Companies UI	64
Figure 5.20: Contact Chatbot UI.....	64
Figure 5.21: Complaint & Suggestion UI.....	64
Figure 5.22: Company Selection UI	64
Figure 5.23:Terms & Conditions UI.....	65
Figure 5.24: FAQs UI	65
Figure 5.25: Refund UI.....	65
Figure 5.26: Refund Process UI.....	65
Figure 5.27: Homepage (Company Admin)	66
Figure 5.28: Payment Details UI	66
Figure 5.29: Route Management UI	66
Figure 5.30: Add Route UI	66
Figure 5.31: Bus Schedule UI.....	67
Figure 5.32: Add Schedule UI	67
Figure 5.33: Manage Tickets UI	67
Figure 5.34: Refund Payment UI.....	67
Figure 5.35: Add Driver UI	68
Figure 5.36: Manage Drivers UI.....	68
Figure 5.37: Add Bus Maintenance UI.....	68
Figure 5.38: Manage Bus Maintenance UI.....	68
Figure 5.39: Company Admin Customer Support UI.....	69
Figure 5.40: Company Admin Complain & Suggestions UI.....	69

List of Tables

Table 1.1: Related System Analysis Contribution.....	8
Table 3.1: Login.....	22
Table 3.2: Registration.....	22
Table 3.3: Booking Tickets.....	22
Table 3.4: Profile Management (User side).....	23
Table 3.5: Payment	23
Table 3.6: Feedback (User Side).....	23
Table 3.7: Notification (User side)	24
Table 3.8: Cancel Ticket	24
Table 3. 9: Refund (User Side)	24
Table 3.10: Login (Company Admin)	25
Table 3.11: Registration (Company Admin)	25
Table 3.12: Refund (Company Admin)	25
Table 3.13: Schedule Management.....	26
Table 3.14: Feedback (Company Admin).....	26
Table 3. 15: Payment (Company Admin).....	26
Table 3.16: Profile Management.....	27
Table 4.1: Data Collection of User Model.....	44
Table 4.2: Data Collection of Booking Model.....	45
Table 4.3: Data Collection of Bus Schedule	46
Table 4.4: Data Collection of Bus Maintenance Model	47
Table 4.5: Data Collection of Chat Model.....	47
Table 4.6: Data Collection of Complaint & Suggestion Model.....	48
Table 4. 7: Data Collection of Drivers Model	48
Table 4.8: Data Collection of Notifications Model	49
Table 4.9: Data Collection of Payment Model	49
Table 4.10: Data Collection of Refund Model.....	49
Table 4. 11: Data Collection of Route Model.....	50

Table 5. 1: External APIs used in Safar	58
---	----

Chapter 1

Introduction

1. Introduction

Safar is a comprehensive bus ticket booking application. It is designed to revolutionize the way passengers interact with bus transportation services by offering a unified digital platform that brings together various bus operators under one roof. The primary aim of the app is to simplify intra city travel by making the entire process from browsing bus schedules to booking a seat convenient, efficient, and user-friendly.

With *Safar*, users can effortlessly search for available buses across different routes, view detailed schedules, compare fares between operators, and choose the best option that suits their travel needs. The app supports a seamless ticket booking process, integrated with secure online payment methods for added convenience. One of the standout features of the app is the gender-based seat selection system, which respects cultural sensitivities and ensures a more comfortable travel experience for all passengers.

In addition to booking, users review their travel history and provide feedback on their journey to help improve service quality. The app maintains a strong focus on user experience, offering a clean and intuitive interface that caters to all age groups.

Furthermore, *Safar* includes a robust company admin portal specifically designed for bus companies. Through this portal, transport providers can register their services, manage their fleet, add or update routes and schedules, view user bookings, and monitor feedback. This dual-platform structure not only enhances passenger experience but also empowers bus companies to digitize and streamline their operations.

By bridging the gap between passengers and bus operators, *Safar* seeks to create a smarter, more reliable transport ecosystem. It aspires to become a go-to solution for anyone looking to travel by bus, promoting accessibility, transparency, and trust in public transport services.

1.1 Brief

The project titled *Safar* is a mobile-based bus ticket booking application aimed at providing a unified digital platform for passengers and bus service providers. The core idea behind the project

is to streamline and modernize the process of booking bus tickets by offering features like real-time schedule viewing, seat selection with gender preferences, online payments, and a dedicated admin panel for transport companies. This app bridges the gap between passengers and multiple bus operators, ensuring a more accessible, efficient, and user-friendly transportation experience.

As a result of the project, a fully functional Android application was developed that allows users to search for buses, select seats, book tickets, make payments online, and receive confirmations instantly. On the backend, bus companies can manage their routes, monitor user activity, and respond to feedback through a dedicated admin interface. The application successfully demonstrates the use of modern mobile technologies to improve public transportation accessibility and management. It fulfills the objective of delivering a reliable and responsive app for both end-users and bus service providers.

The development of *Safar* involved several tools and technologies. Flutter was used as the main development framework due to its cross-platform capabilities and flexible UI components, with Dart serving as the programming language. The firebase was chosen for the backend, providing real-time database services, user authentication, and cloud storage. The app was developed using Android Studio, a powerful IDE for Flutter development. For diagrammatic representations such as flowcharts and system architecture, Draw.io was utilized. MS Word was used for writing and formatting the project documentation and report.

The project followed a structured and iterative development methodology based on the agile approach. The development was divided into sprints, where key features were planned, implemented, tested, and refined. Initial stages involved requirement gathering and system design, followed by frontend and backend integration. Continuous testing and debugging ensured the app's stability and performance. User feedback was also considered to enhance the interface and features. The Agile approach allowed flexibility in development, enabling incremental progress and timely adjustments throughout the project lifecycle.

The report begins with an introduction to the problem domain and the need for a centralized bus booking system. The following chapters cover a detailed review of existing systems and their limitations, establishing the motivation for *Safar*. The design and implementation chapters provide insights into the architecture, UI design, backend integration, and system flow. The testing and evaluation chapter discusses performance results, usability feedback, and error handling

mechanisms. The report concludes with a summary of key achievements, challenges faced, and recommendations for future improvements [1].

1.2 Relevance to Course Modules

This relevance of this app to the courses we have learnt in the degree is,

- **Software Engineering Concepts**

This course provided a foundational understanding of the software development lifecycle, including requirement gathering, analysis, design, implementation, testing, and maintenance. It helped structure the Safar app development in a systematic way, ensuring clear documentation, modular design, and quality assurance throughout the project. The knowledge gained also facilitated better collaboration within the team, promoting the use of agile practices and iterative development. Overall, it laid a strong groundwork for managing the project's complexity and meeting user expectations effectively.

- **Object Oriented Programming (OOP)**

OOP introduced the concepts of classes, objects, inheritance, encapsulation, and polymorphism, which were vital in structuring the codebase of the Safar app. These principles allowed us to build reusable, maintainable, and scalable components, particularly for features like ticket management, user accounts, and admin functionalities. The modular approach made it easier to debug and extend the system as new features were added. It also improved code readability and collaboration among developers by promoting a clear separation of concerns [2].

- **Mobile Application Development**

This course is directly aligned with the project goals, equipping us with the skills needed to design, develop, and deploy mobile applications. It covered essential topics such as UI/UX design, platform-specific considerations, and backend integration, which were crucial in building the Safar app using Flutter and Firebase. The hands-on assignments and projects provided practical experience that mirrored real-world app development scenarios. Additionally, the course emphasized performance optimization and responsive design, ensuring a smooth user experience across various devices. The knowledge gained greatly contributed to building a functional, user-friendly, and robust mobile application. [3].

- **Software Design Methodologies**

Through this course, we explored various design models such as MVC, layered architecture, and component-based design. These methodologies were applied in the project to ensure a clean separation of concerns, better organization of code, and smoother integration between frontend and backend components. Applying these models enhanced the maintainability and scalability of the Safar app. Moreover, it allowed different team members to work on distinct layers simultaneously, improving development efficiency and collaboration [4].

- **Data Structures and Algorithms**

This course provided the analytical thinking and technical skills needed to build efficient and responsive software. We used various data structures like lists, maps, and queues to manage data effectively within the app, especially in handling seat selection, schedule display, and booking operations. Understanding the time and space complexity of these structures helped us make performance-optimized decisions. This foundation ensured that the app remained responsive even when processing large volumes of real-time data.

- **Software Project Management**

This course taught key project management principles like project planning, risk assessment, time management, and resource allocation. It helped us manage the Safar app development efficiently by defining clear milestones, managing tasks, and delivering a working product within the project timeline. The use of project management tools improved team coordination and progress tracking. Additionally, regular evaluations and updates helped in identifying potential risks early and adapting plans accordingly.

- **Human Computer Interaction (HCI)**

HCI focused on designing user-friendly and accessible interfaces. The knowledge from this course was applied to improve the usability and aesthetics of the Safar app, ensuring both passengers and bus operators could interact with the system easily and intuitively. We implemented consistent design elements like color schemes, typography, and layouts to maintain a smooth user experience. Key principles such as feedback, visibility, and error prevention were used to guide users through core tasks like booking and schedule viewing. Accessibility considerations, including touch-friendly controls and proper contrast, made the

app usable for a wider audience. Usability testing and user feedback further helped us refine the design, resulting in a more effective and inclusive interface. [5].

1.3 Project Background

In today's fast-paced world, the demand for convenient, accessible, and real-time travel solutions has significantly increased. Traditional bus ticket booking systems often rely on manual processes, physical counters, and limited access to schedules and seat availability, which can be inefficient and time-consuming. The concept of digital ticket booking platforms emerged to address these issues by centralizing information and offering passengers more control over their travel planning.

There are other digital booking systems present too, some lack interactive designs, i.e., the user interface is too clustered, and some of these systems are not solely dedicated to bus ticket booking. Some of these systems also lack company admin portals and many other features included in this app.

Inspired by global advancements in mobile technology and online booking systems, the *Safar* app was developed as a solution tailored to local transportation needs. It integrates multiple bus services into a single platform, allowing users to easily search, compare, and book tickets, while enabling bus operators to manage their services digitally. The project builds upon modern mobile application frameworks, cloud-based databases, and user-focused design to improve the overall travel experience.

With the rise of smartphones and digital platforms, passengers now expect greater flexibility, real-time updates, and seamless integration with various service providers. However, existing systems often lack the personalization and ease-of-use required for an optimal customer experience. The *Safar* app addresses these gaps by offering a user-friendly interface and providing bus companies with a comprehensive admin portal to manage their services efficiently. This innovative solution not only streamlines the booking process but also ensures smoother coordination between passengers and service providers, setting a new standard for digital travel management.

1.4 Literature Review

In the existing system analysis, we look for apps which match the working or features of our system. We analyze these systems deeply to learn the user experience and feedback related to that system and then overcome or reduce the deficiencies of that system in our application.

1.4.1 Existing System Analysis

1. Bookme.pk

- **Description**

The **Bookme** app is a leading online ticketing platform in Pakistan, offering users the ability to book tickets for buses, flights, cinemas, hotels, and events. Established in 2013 and headquartered in Lahore, it has transformed the way people secure tickets, providing a convenient, transparent, and efficient solution. With partnerships with over 150 bus operators covering major intercity routes, as well as collaborations with airlines and event organizers, Bookme offers a wide range of services. The app features a user-friendly interface that allows users to browse schedules, view seating plans, and make secure payments. It also offers various discounts and promotional deals, such as up to 30% off on bus tickets with select operators.

- **Proposed Solution**

Our app is easy to use, less complex and offers a more user-friendly interface.

2. EasyTickets

- **Description**

EasyTickets is a leading online ticketing platform in Pakistan, offering a comprehensive solution for booking bus, cinema, and event tickets. With partnerships with over 25 bus operators, 30 cinemas, and major event organizers, EasyTickets provides a user-friendly interface for users to search, compare prices, and purchase tickets digitally. All transactions are processed digitally, and tickets are delivered instantly via QR code, email, and SMS, eliminating the need for physical delivery or cash-on-delivery methods.

- **Proposed Solution**

Our app will offer comprehensive bus schedules and flexible UI for users, a smoothly operating interface and no problems in login and signup.

3. Daewoo Express

- **Description**

Daewoo Express is a premier intercity bus service in Pakistan, renowned for its commitment to luxury, safety, and punctuality. Established in 1997 and headquartered in Lahore, it operates a fleet of over 400 buses, connecting more than 60 destinations nationwide, including major cities like Karachi, Islamabad, Lahore, and Peshawar. The company offers a range of services, from economy to business class, ensuring comfort and reliability for travelers. In addition to passenger services, Daewoo Express provides cargo solutions through its subsidiary, Daewoo Fastex, and operates urban transit systems in cities like Lahore and Multan. With a focus on modern infrastructure and customer satisfaction, Daewoo Express continues to be a leader in Pakistan's transportation sector.

- **Proposed Solution**

We will make a simple and aesthetic design which would not be complex. It will have an intuitive interface, and various payment methods for greater flexibility.

1.5 Analysis from literature review

The analysis of the like systems reveals critical gaps such as limited real-time functionality, lack of intuitiveness, ease, and restricted user choice due to service exclusivity.

Our proposed app addresses these issues by a user-friendly and simple interface, which would make the app accessible and easy to use for a broader demographic and by including multiple bus agencies and versatile payment options, our app provides users with the freedom to choose services that fit their needs best, alongside many enhanced features.

The system also emphasizes scalability and performance to ensure smooth operation during peak travel seasons.

Security protocols are integrated to protect user data and ensure secure transactions.

In-app support and feedback mechanisms allow users to quickly resolve issues and suggest improvements. Overall, the app offers a unified platform that simplifies travel while maximizing flexibility and reliability.

Table 1.1: Related System Analysis Contribution

Application Name	Weakness	Proposed Project Solution
Bookme.pk	Issues with real-time updates, user interface.	User-friendly interface, tailored features for users.
EasyTickets	Lacks bus-specific features, lagging, sign up issues.	Detailed agency profiles, comprehensive schedules, no lagging.
Daewoo Express	Limited to own services, lacks fare comparison and refund issues.	Multi-agency integration, fare comparison, and a robust feedback system.

1.6 Methodology and Software Development Life Cycle

1.6.1 Methodology and SDLC Selected

The development of the *Safar* bus ticket booking application followed the **Agile Software Development Methodology**. Agile is an iterative and incremental approach that emphasizes flexibility, collaboration, and continuous improvement. The Agile model breaks the project into manageable units called "sprints," each focusing on specific features or tasks. At the end of each sprint, a working version of the product is produced, tested, and evaluated. This allowed us to gather feedback early and adjust development based on evolving requirements [6].

In terms of the **Software Development Life Cycle (SDLC)**, the project followed a **modified Agile SDLC** that included the following stages:

- **Requirement Gathering & Analysis:** Understanding the needs of passengers and bus operators.
- **Design:** Creating system architecture, UI mockups, and planning backend structure.
- **Implementation:** Developing the app using Flutter (Dart) and integrating Firebase for backend services.

- **Testing:** Conducting functionality, usability, and performance testing in each sprint.
- **Deployment:** Releasing the app in a test environment and preparing it for real-world use.
- **Maintenance:** Fixing bugs and refining features based on feedback.

This lifecycle allowed us to remain adaptable and ensured that all features—from seat selection to admin panel were developed, tested, and improved incrementally.

1.6.2 Rationale behind selecting methodology

We chose the **Agile methodology** for this project due to its **flexible and adaptive nature**, which perfectly matched the dynamic scope of the *Safar* app. Since we were working on multiple features (like user bookings, payments, and admin management) with evolving ideas and requirements, Agile allowed us to break the project into smaller, manageable tasks and improve them with each sprint. It also encouraged regular team communication and early detection of problems, improving both development speed and software quality.

In terms of programming structure, an **Object-Oriented approach** was selected because it promotes modular and reusable code. Using OOP in Dart allowed us to design the app around real-world entities such as User, Bus, Booking, and Admin, making the system easier to manage, debug, and expand. The combination of agile methodology and OOP principles enabled a clean, organized, and efficient development process.

Chapter 2

Problem Definition

2. Problem Definition

This section outlines the specific problem to be addressed and its anticipated outcome:

2.1 Problem Statement

In Pakistan, intercity bus travel is a widely used mode of transportation, but existing systems for booking tickets are often fragmented, outdated, and lack standardization across various travel companies.

Customers face inconvenience due to inefficient ticket booking processes, limited payment options, poor seat selection mechanisms, and lack of real-time support. Moreover, bus companies often lack an organized platform to manage schedules, driver details, ticket availability, and user complaints effectively.

The absence of integration between different bus operators also leads to passengers needing to check multiple platforms to compare prices, routes, and schedules. As a result, the overall travel experience is often cumbersome and frustrating for both travelers and service providers.

Furthermore, the reliance on manual systems and limited digital infrastructure means that bus companies struggle to optimize their operations, leading to delays, errors, and a lack of coordination. There is a pressing need for a centralized digital solution that connects users with multiple bus companies under a unified platform, offering a smooth and secure ticket booking experience.

This platform would improve operational efficiency, enhance customer satisfaction, and bring modernization to the bus travel industry in Pakistan.

2.2 Problem Solution

The proposed system, **Safar**, is a mobile application that bridges the gap between passengers and transport service providers by providing a centralized platform for intercity bus ticket booking. The app allows different transport companies to register and manage their routes, schedules, buses, and drivers through a dedicated company admin panel, which is verified and monitored by a Main Admin. Users can register, browse available routes, filter buses based on preferences, select

gender-based seats, and securely make payments through integrated local payment gateways like JazzCash and EasyPaisa. The system also offers ticket history, refunds, complaint handling, and customer support via chatbot, ensuring a convenient and reliable travel booking experience.

In addition to these features, Safar enables real-time updates on bus schedules, delays, and cancellations, allowing passengers to plan their journeys with greater accuracy. The app also provides users with the option to save favorite routes and buses, making future bookings faster and more efficient.

A complaints and suggestions system allows passengers to provide feedback on their travel experience, helping other users make informed decisions. Bus operators can monitor their fleet's performance, including seat occupancy and revenue, ensuring more effective management of their services. The app will also include notifications for special offers, discounts, and promotions to enhance user engagement.

By integrating a robust search feature, users can easily locate available buses, compare fares, and select the most convenient options for their travel needs. The system's secure and scalable infrastructure ensures smooth operation, even during peak travel times. Ultimately, Safar aims to revolutionize the intercity bus booking experience by providing a seamless, user-friendly, and efficient platform for both passengers and transport providers.

2.3 Objectives of the Proposed System

- To develop a centralized platform for multiple intercity bus companies to register and manage their services. Bus companies can create accounts and are verified by the main admin before accessing their company dashboard. Through this panel, they can add and manage routes, schedules, buses, and drivers. This centralized system allows multiple companies to operate under one platform, providing a wide range of options to users.
- To facilitate users with an easy-to-use mobile app for browsing, filtering, and booking bus tickets. The user app is designed with a simple interface where users can search for available buses by selecting departure and arrival cities along with travel dates. After applying filters for bus class and company, users can view available trips with full details like time, price, services, and seat availability, and proceed to book tickets seamlessly.

- To provide secure and localized payment methods (JazzCash and EasyPaisha) with proof of payment functionality. After selecting seats and confirming booking, users are guided to send payment through JazzCash or EasyPaisha. They must upload a screenshot and enter the transaction ID, which is verified by the company admin to confirm the ticket. This manual verification adds a security layer and prevents scams or fake bookings.
- To implement gender-sensitive seat selection for passenger safety and comfort. The app shows a seat layout with color-coded indicators: pink for female, blue for male, yellow for selected, and green for family bookings. Users are prompted with a warning if they try to book a seat next to someone of the opposite gender, unless they choose the multiple-seat (family) option. This ensures respectful and comfortable seating arrangements for all passengers.
- To offer admin panels for company and main administrators to manage schedules, drivers, buses, complaints, and refunds. Company admins can update travel details, add drivers, view and respond to complaints, and handle refund requests. The main admin oversees all company registrations, approves new entries, monitors ticket activity, and ensures smooth operation of the entire system.
- To include features such as booking history, real-time notifications, refund processing, and a chatbot for user queries. Users can track their ticket status under the booking history tab, from pending to confirmed or refunded. Notifications alert them about booking approvals or changes. The refund feature allows users to request and receive payments securely, and a built-in chatbot offers 24/7 assistance for common questions or issues.

2.4 Scope

The scope of SAFAR encompasses both the user-facing mobile application and administrative web interfaces. The app supports:

- User registration, login, password recovery.
- Viewing available routes and buses.
- Gender-based seat selection and ticket booking.
- Secure payments through JazzCash and EasyPaisha.
- Ticket tracking, cancellation, and refund.

- Profile and history management.
- Chatbot and feedback system.

On the administrative side, it includes:

- A **Main Admin Panel** for approving registered companies.
- A **Company Admin Panel** for managing payments, schedules, drivers, buses, customer messages, and processing refunds.

The project focuses on intercity travel within Pakistan, covering multiple bus service providers under one umbrella.

2.5 Modules

The following are the modules of Safar:

2.5.1 Module 1: User Profiling

This module is responsible for managing the identity, access, and personal information of both users and companies within the SAFAR system. It includes user registration, login functionality (including Google sign-in), password recovery, and profile management. It ensures that only authenticated users can access the system, and that personal data is stored and updated securely.

- **Signup:** Enable users and bus companies to create accounts by providing required details such as name, email, phone number, and password. Data is stored securely in the database. Separate registration options for general users and companies.
- **Login:** Authenticates users, company admins, and the main admin. Provides three access levels: User Side, Company Admin Side, and Main Admin Side. Includes "Login with Google" option for quick access.
- **Forget Password:** Allows users to recover access by entering their registered email. Send a password reset email with a secure link.
- **Profile Management:** Displays and allows editing of user details like name, profile picture, phone number, and email. Ensure users can keep their personal information updated.

2.5.2 Module 2: Ticket Booking

This module handles the entire process of booking a bus ticket. It allows users to search for routes, filter available buses, select seats based on gender preferences, confirm bookings, and make payments. The goal is to provide a smooth, user-friendly, and secure ticketing experience from selection to confirmation.

- **Route Selection:** Users select the departure city, arrival city, and date of travel. Validates and fetches relevant routes from verified companies.
- **Filter and Bus Selection:** Displays a filter popup listing all companies available on the selected route. Includes filtering options for Business and Economy class. Lists tickets with route details, terminals, timing, price, services (Wi-Fi, dinner), company name, and travel duration.
- **Seat Selection:** Gender-based seat selection with color-coded seats. Toast messages for gender-related seat conflicts.
- **Ticket Confirmation:** Displays full ticket details including seat number, class, route, time, and cancellation policy. Includes Confirm Booking button.
- **Payment Method:** Choose payment method: JazzCash or EasyPaisa. Enter payment details and upload screenshots. Ticket marked as "Pending" until verified by admin.
- **Booking History:** View all booked tickets with status (Pending, Approved, Cancelled). Cancel ticket or request refund.

2.5.3 Module 3: Admin Panel (Company Admin)

This module is for bus companies to manage their operations within the SAFAR system. After being verified by the Main Admin, companies use this panel to add payment options, manage routes, schedules, buses, drivers, process ticket approvals, and handle customer interactions and refunds.

- **Company Registration and Verification:** Company fills out a registration form. Pending status until approved by Main Admin. Only verified companies appear in the user app.
- **Payment Setup:** Company Admin adds JazzCash and EasyPaisa numbers for receiving payments.

- **Route and Schedule Management:** Create and manage bus routes and schedules. Set departure/arrival cities, terminals, times, prices, and available classes.
- **Driver and Bus Management:** Add and maintain driver profiles and assign them to buses. Manage fleet information including bus capacity and services.
- **Ticket Approval:** View payment screenshots and TIDs. Approve or reject bookings. Change booking status from "Pending" to "Approved."
- **Refund Management:** Process refund requests by verifying details and transferring amount via payment method.
- **User Feedback and Chatbot:** View suggestions and complaints. Interact with users via chatbot.

2.5.4 Module 4: Admin Panel (Main Admin)

The Main Admin module is designed for platform-level supervision and control. The Main Admin oversees company registrations and verifies companies.

- **Company Verification:** Reviews and approves/rejects new company registration requests. Only verified companies can access the Company Admin Panel.
- **Monitoring and Control:** Monitors all companies, bookings, complaints, and refund activities.

2.5.5 Module 5: Additional Features

This module includes all supplementary but essential features that improve user engagement, support, and transparency in the app. It includes a chatbot, notifications, a navigation drawer, and access to legal and informational content.

- **Chatbot:** Interactive chatbot that provides 24/7 help for common queries. Connect users with relevant company admin when needed.
- **Notifications:** Send alerts for booking status, refunds, and important updates.
- **Drawer Menu:** Includes navigation to Suggestions, Complaints, Contact Company, Refunds, Terms & Conditions, FAQs, and Logout.

2.6 Deliverable and Development Requirements

2.6.1 Deliverables

- A fully functional Android mobile application (*Safar*) with user interface for passengers to book tickets, view history, apply for refund, manage their profile (which includes changing email, uploading profile picture, changing password), complain and make suggestions about the services of a certain bus service etc.
- Admin interface for bus operators to manage schedules, refunds, drivers, bus maintenance, user feedback, bus tracking etc.
- Firebase-based backend for authentication, data storage, and real-time updates.
- Complete documentation including design diagrams, testing reports, and user manuals.

2.6.2 Development Requirements

- **Platform:** Flutter (for cross-platform mobile development)
- **Language:** Dart
- **Backend Services:** Firebase (Authentication, Firestore, Supabase, Emailjs)
- **IDE:** Android Studio
- **Diagram Tools:** Draw.io
- **Documentation Tools:** MS Word
- **Hardware:** Android smartphone (for testing), PC/Laptop

Chapter 3

Requirement Analysis

3. Requirement Analysis

This section describes the techniques that identify Safar requirements in such a way that it maps a way for better understanding of its functional and non-functional requirements:

3.1 Requirement Identifying Technique

3.1.1 Use Case Diagram

The figures 3.1, 3.2, 3.3 and 3.4 show the common, user, admin and company admin use case diagrams for safar app.

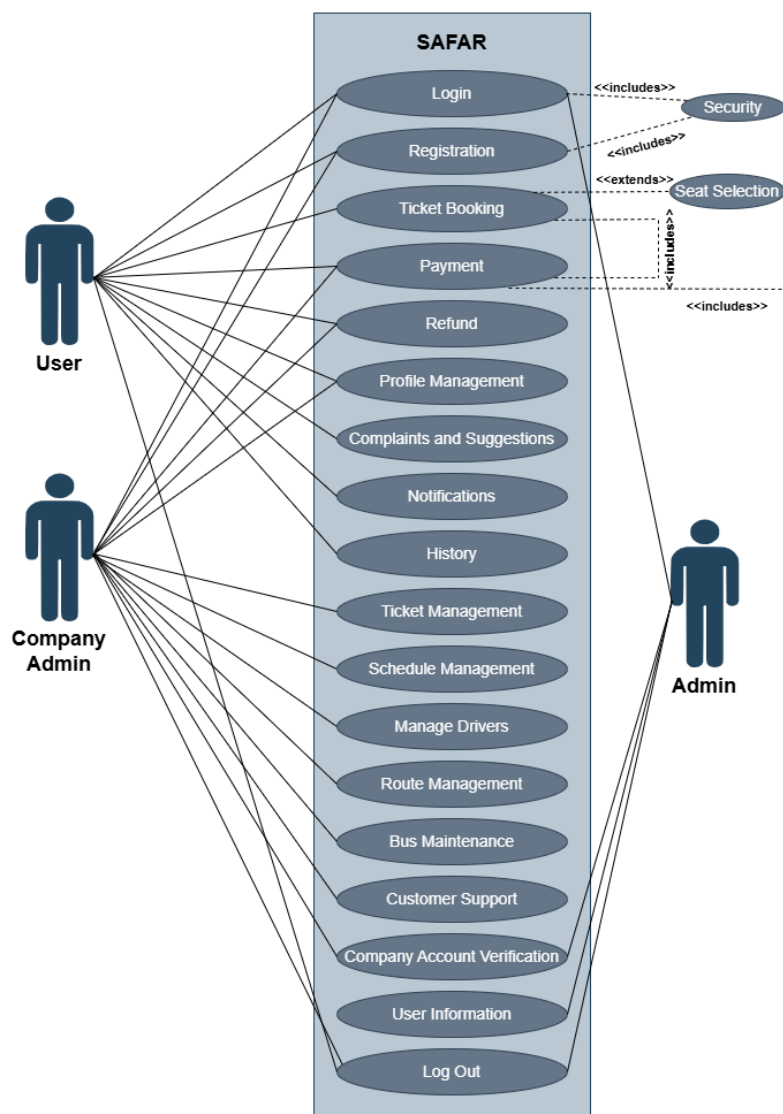


Figure 3.1: Common Use Case Diagram

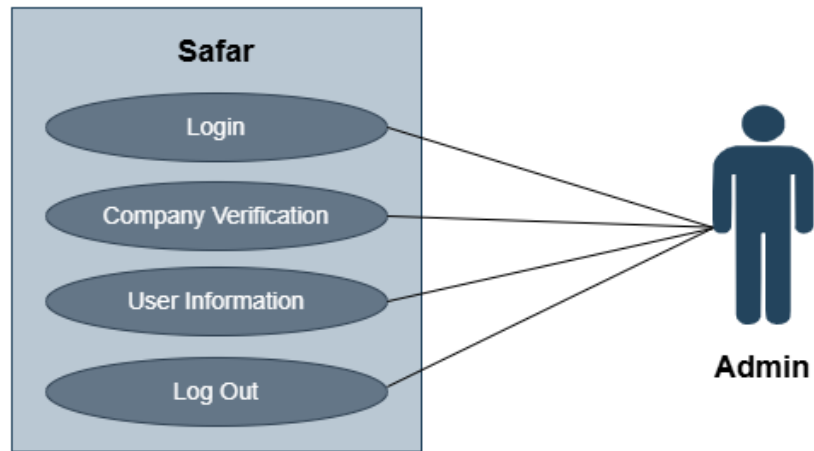


Figure 3.2: Admin Use Case Diagram

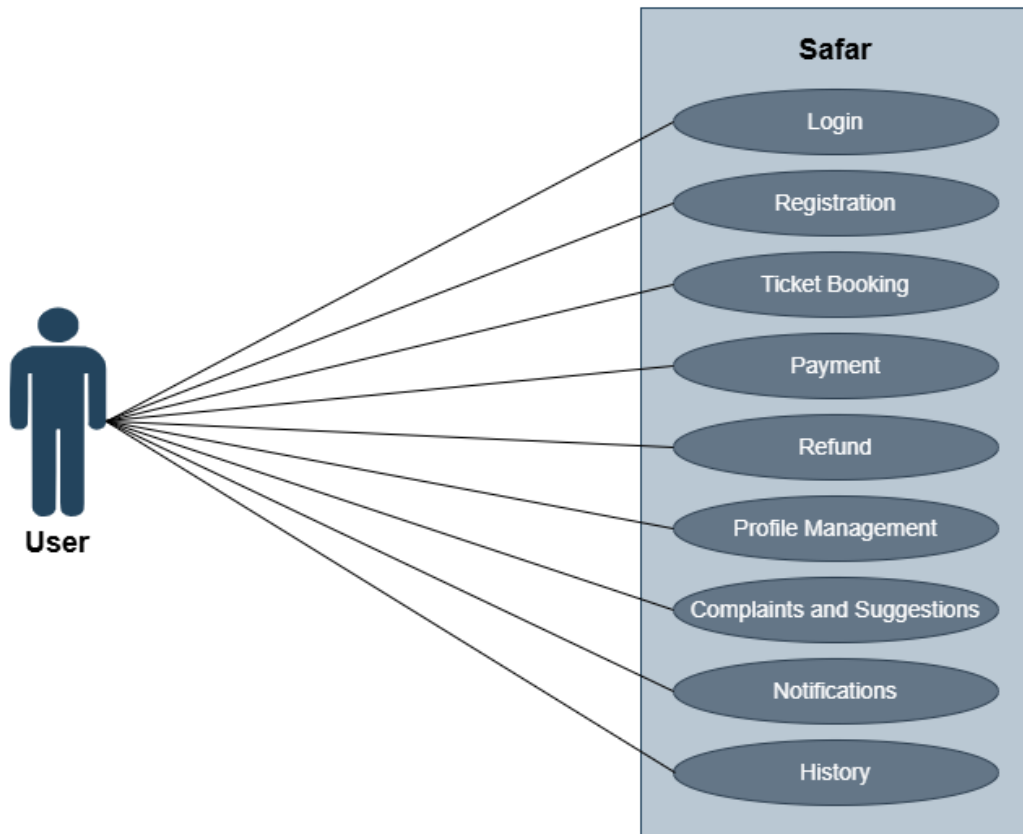


Figure 3.3: User Use Case Diagram

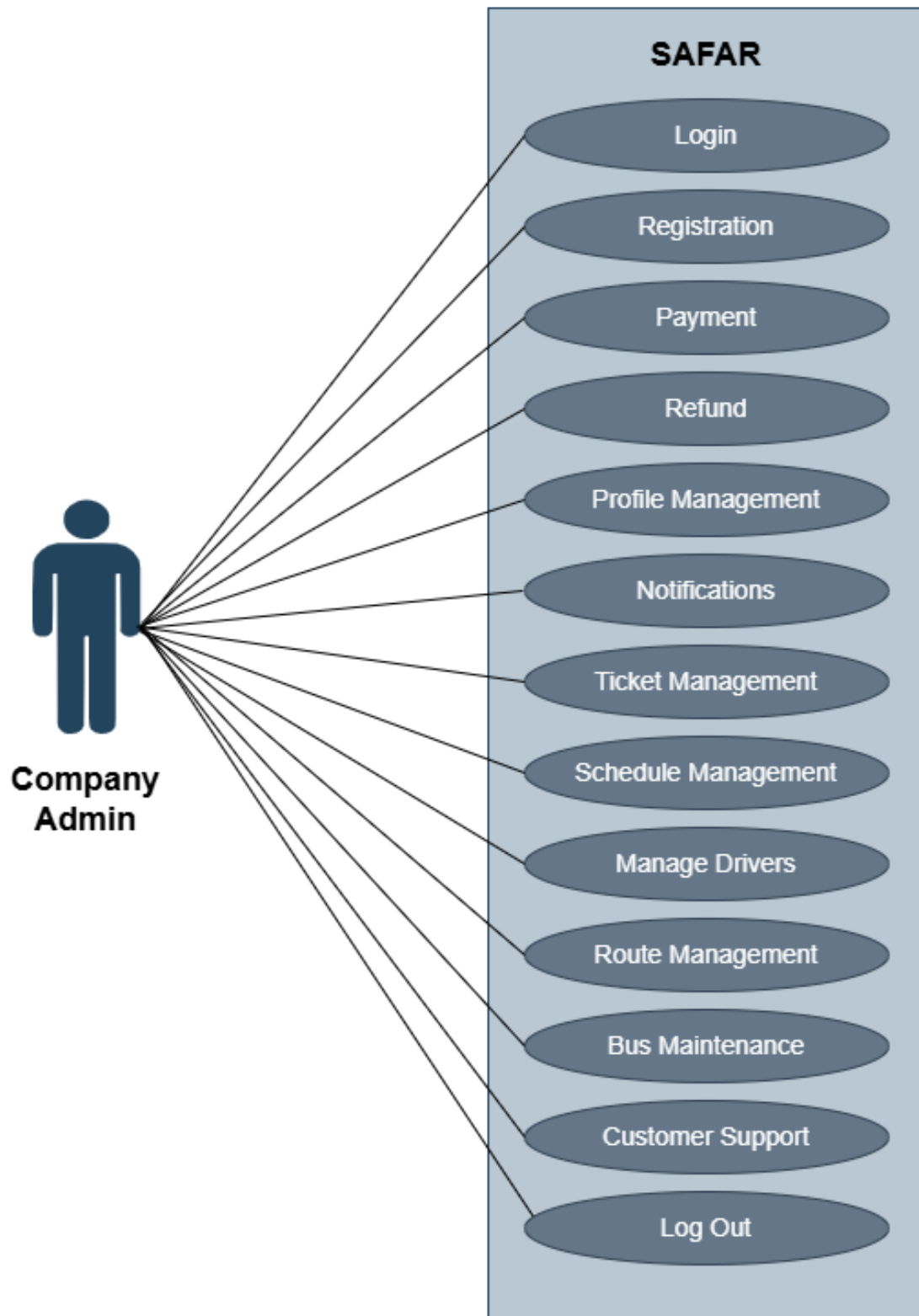


Figure 3.4: Company Admin Use Case Diagram

3.1.2 Tabular Use Cases

Here are the use case descriptions for Safar:

Table 3.1: Login

Use Case	Login
Primary-Actor	User
Goal	To access the app's features by logging in
Pre-conditions	User has an account
Triggers	User clicks on the login button
Description	The system logs in to the user after verifying their credentials.
Post-conditions	User is logged in and can access app features

Table 3.2: Registration

Use Case	Registration
Primary Actor	User
Goal	To create an account to use the app's features
Pre-conditions	User does not have an account
Triggers	User clicks on the register button
Description	The user enters their details, and the system creates an account.
Post-conditions	User has an account and can log in

Table 3.3: Booking Tickets

Use Case	Booking Tickets
Primary Actor	User
Goal	To book a ticket for a selected route
Pre-conditions	User has selected a route and upload payment proof
Triggers	User clicks on the book ticket button
Description	The system processes the booking after verifying the payment
Post-conditions	Ticket is booked, and user receives a booking confirmation

Table 3.4: Profile Management (User side)

Use Case	Profile Management
Primary Actor	User
Goal	To view details of the profile
Pre-conditions	User login
Triggers	Click different option buttons
Description	The system displays the credentials and changes password option
Post-conditions	User can manage profile

Table 3.5: Payment

Use Case	Payment
Primary Actor	User
Goal	Paying for the ticket
Pre-conditions	Ticket is booked
Triggers	System gives options for online wallet payments.
Description	The system verifies the payment against TID
Post-conditions	User receives booking confirmation

Table 3.6: Feedback (User Side)

Use Case	Feedback
Primary Actor	User
Goal	To provide feedback about the app or service
Pre-conditions	User has used the app's features
Triggers	User clicks on the complaint and suggestion button
Description	The user enters their feedback, and the system stores it.
Post-conditions	Feedback is stored for future improvement

Table 3.7: Notification (User side)

Use Case	Notifications
Primary Actor	User
Goal	To view notifications
Pre-conditions	User login
Triggers	Click notification button on nav bar
Description	The system displays the notifications
Post-conditions	Users can view updates via notifications

Table 3.8: Cancel Ticket

Use Case	Cancel Ticket
Primary Actor	User
Goal	To cancel ticket
Pre-conditions	Ticket booked
Triggers	Go to Bookings on nav bar
Description	The system cancels the ticket, and the refund screen open
Post-conditions	Refund the ticket

Table 3. 9: Refund (User Side)

Use Case	Refund
Primary Actor	User
Goal	To refund ticket
Pre-conditions	Cancel Ticket
Triggers	Click on Cancel Ticket button
Description	The system displays the refund options
Post-conditions	Users can apply for refund

Table 3.10: Login (Company Admin)

Use Case	Login
Primary-Actor	Company Admin
Goal	To access the app's features by logging in
Pre-conditions	Company Admin has an account
Triggers	Company Admin clicks on the login button
Description	Company Admin enters their credentials, and the system verifies them. If valid, the user is logged in.
Post-conditions	Company Admin is logged in and can access app features

Table 3.11: Registration (Company Admin)

Use Case	Registration
Primary Actor	Company Admin
Goal	To create an account to use the app's features
Pre-conditions	Company Admin does not have an account
Triggers	Company Admin clicks on the register button
Description	Company Admin enters their details, and the system creates an account.
Post-conditions	User has an account and, but verification is to be completed

Table 3.12: Refund (Company Admin)

Use Case	Refund
Primary Actor	Company Admin
Goal	To refund ticket of user
Pre-conditions	Ticket is cancelled by user
Triggers	Click on Refund the ticket button
Description	The system displays the refund options
Post-conditions	User gets their refund

Table 3.13: Schedule Management

Use Case	Schedule Management
Primary Actor	Company Admin
Goal	To publish a schedule
Pre-conditions	Add route
Triggers	Click on Bus Schedule
Description	The system displays the add schedule options
Post-conditions	A schedule is published

Table 3.14: Feedback (Company Admin)

Use Case	Feedback
Primary Actor	Company Admin
Goal	To provide and check feedback about the app or service
Pre-conditions	User has given some feedback
Triggers	Company Admin clicks on the customer support button
Description	The company admin checks the feedback and responds to it.
Post-conditions	Feedback is stored for future improvement.

Table 3. 15: Payment (Company Admin)

Use Case	Payment
Primary Actor	Company Admin
Goal	Check payments
Pre-conditions	Ticket is booked; payment has been made by a user
Triggers	System gives options for online wallet payments.
Description	The system sends a confirmation message to the admin.
Post-conditions	User receives booking confirmation notification.

Table 3.16: Profile Management

Use Case	Profile Management
Primary Actor	Company Admin
Goal	To view details of the profile
Pre-conditions	Company Admin login
Triggers	Click different option buttons
Description	The system displays settings like change password, update profile picture etc.
Post-conditions	Company Admin can manage profile

3.2 Functional Requirements

This section outlines the functional requirements of the system, which specify the core features and operations that the system must be able to perform. These include user interactions, data input and output, processing rules, and specific tasks the system should handle. Functional requirements ensure that the system behaves as expected and fulfills the objectives of the project [7].

3.2.1 Ticket Booking Module

This module allows users to search for buses, select seats, and book tickets. It integrates with the schedules of various bus agencies, allowing users to compare options and choose the best one for their needs.

3.2.2 Online Payment Module

Facilitates secure online transactions for ticket payments. Users can mobile wallets. The module will be designed with an emphasis on security, ensuring all user data and payment information is encrypted and securely processed.

3.2.3 Bus Schedule and Fare Comparison Module

Integrates bus schedules from various agencies into the app, providing users with comprehensive timetable information. This module also allows users to compare fares for different buses or routes with the help of filters, helping them make cost-effective travel decisions based on their schedules and budget.

3.2.4 Feedback and History

Users can submit feedback directly through the app, influencing future updates and improvements. The user can make suggestions, complain and contact the company.

3.3 Non-Functional Requirements

This section outlines the non-functional requirements, which define the quality attributes and constraints of the system. These include aspects such as performance, reliability, scalability, security, and usability. While they do not specify particular functions, non-functional requirements are essential to ensure that the system operates efficiently and delivers a satisfactory user experience [8].

3.3.1 Usability

The system should be easy to use and navigate, with an intuitive interface that allows users to quickly find and book their desired routes. The design should prioritize simplicity, ensuring that even first-time users can understand and use the app without confusion. Clear visual cues, such as well-labeled buttons and organized sections, will guide users through each step of the booking process, from selecting routes to completing payment. The interface should also be responsive, offering seamless navigation across different devices and screen sizes. By focusing on user experience, the app will minimize friction, enabling passengers to make bookings with minimal effort and time. This approach will help enhance customer satisfaction and foster trust in the app's reliability and ease of use.

3.3.2 Performance

The system should respond quickly to user input, with a maximum response time of 2 seconds for most transactions, ensuring a smooth and efficient user experience. To achieve this, the app will be optimized for performance, employing techniques like data caching and load balancing to reduce latency. The system should also be able to handle a minimum of 100 concurrent users without compromising speed or stability, even during peak usage times. Scalability is key, so the infrastructure will be designed to accommodate future growth in user traffic and ensure reliability under heavy load. Regular performance testing and monitoring will be implemented to identify and resolve potential bottlenecks, maintaining optimal performance as the app scales. This will provide users with a seamless, uninterrupted experience, encouraging continued usage of the platform.

3.3.3 Security

The system should ensure the confidentiality, integrity, and availability of user data, with encryption of sensitive information and secure authentication and authorization mechanisms. Data at rest and in transit should be encrypted using industry-standard protocols, safeguarding user information from unauthorized access. Multi-factor authentication (MFA) will be implemented for added security, ensuring that only authorized users can access their accounts. Regular security audits and vulnerability assessments will be conducted to identify potential risks and ensure that the system remains secure against evolving threats. Additionally, user data will be stored in compliance with relevant data protection regulations, maintaining trust and transparency [9].

3.3.4 Portability

The system should be accessible on a variety of devices, including desktop computers, laptops, tablets, and mobile phones, with a responsive design that adapts to different screen sizes and resolutions. The user interface should automatically adjust to ensure a consistent and seamless experience across all platforms, whether users are accessing the app via a large desktop monitor or a smaller mobile screen. Touchscreen functionality should be optimized for tablets and smartphones, allowing users to interact easily with the app's features. The design will prioritize legibility and usability, ensuring that fonts, buttons, and other interactive elements remain clear and functional regardless of the device used.

3.3.5 Reliability

The system should be available 99.9% of the time, ensuring minimal downtime and uninterrupted service for users. High availability will be achieved through robust infrastructure, redundancy, and load balancing to ensure that any potential issues are quickly addressed. In the event of a failure, the system should be able to recover quickly, with automatic failover mechanisms and backup systems in place to minimize disruption. The system should also be intuitive and user-friendly, with a straightforward interface and easy navigation to ensure that users can quickly access and utilize the app's features without unnecessary complexity.

3.3.6 Maintainability

The system should be easy to maintain and update, with a modular design that allows for easy modification and replacement of individual components. This modular architecture will enable developers to quickly implement new features, fix bugs, or update existing functionalities without

affecting the entire system. Each module should be independently deployable, ensuring that updates can be rolled out efficiently with minimal downtime. Additionally, comprehensive documentation will be provided to streamline the maintenance process, making it easier for the development team to manage the system over time.

3.3.7 Scalability

The system should be able to handle increased traffic and user activity, with the ability to scale up or down as needed to meet changing demands. This will be achieved through cloud-based infrastructure that supports auto-scaling, ensuring resources are allocated dynamically based on real-time traffic patterns. Load balancing techniques will distribute traffic efficiently across multiple servers, preventing bottlenecks and ensuring consistent performance even during peak usage. The system should also be capable of scaling to accommodate future growth, with the flexibility to expand in response to rising user numbers or additional features [10].

Chapter 4

Design and Architecture

4. Design and Architecture

This section outlines the system's structural framework based on the Software Design Specification (SDS), including architectural design, design models, and data design.

4.1 System Architecture

Safar app follows a **MVC (Model, View, and Controller)** design pattern. The Model handles data manipulation logic and the View presents this information in an accessible format. Serving as an intermediary, the Controller interprets user inputs from the View and communicates with the Model, ensuring a seamless and organized user experience. This architecture enhances maintainability and scalability, allowing modifications in one aspect without disrupting others. Users engage with the application through View, with the Controller facilitating smooth interactions and data flow in our Safar app [11].

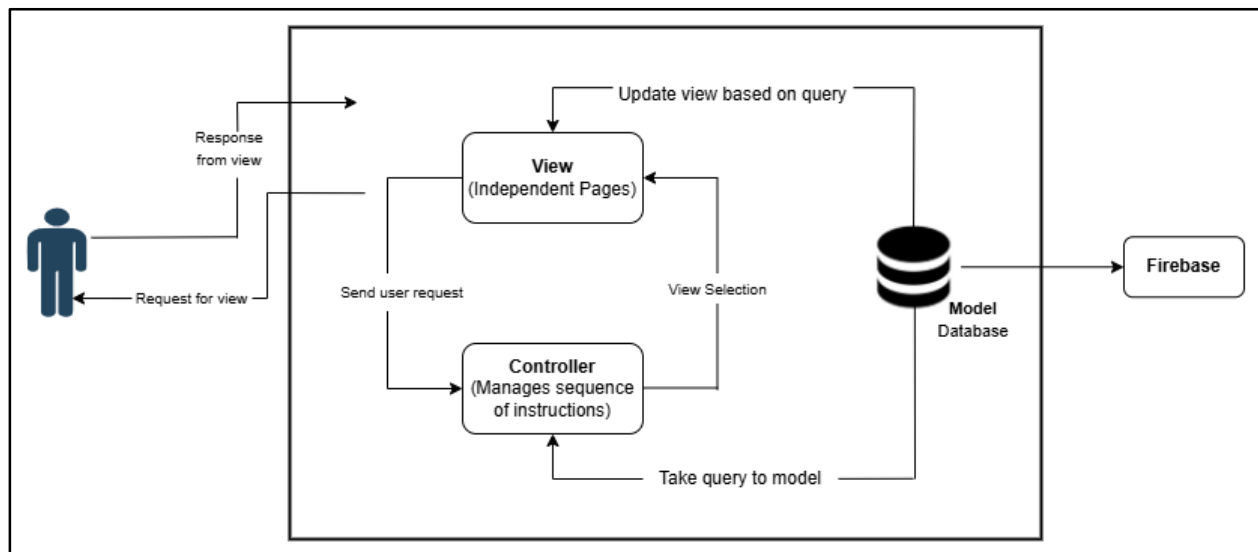


Figure 4. 1: MVC System Architecture Diagram

4.2 Design Models

The design models for our application which uses Object Oriented Approach are as below:

4.2.1 Activity Diagram

The following are the activity diagrams of our system:

An **activity diagram** is a visual representation of the flow of activities within a system, often used in software engineering to model the dynamic aspects of a system. It is part of the Unified Modeling Language (UML) and helps illustrate how a particular process or workflow progresses from one step to another. The diagram includes various elements such as the initial node (starting point), activities (actions or tasks), decisions (represented by diamond shapes to indicate branching paths), control flows (arrows showing sequence), and a final node (ending point). Activity diagrams are particularly useful for describing complex use cases, business processes, and user interactions with a system [12].

Figure 4.2 shows the activity diagram for Safar. The process begins with a decision point that checks whether the account already exists. If the account does not exist, the user is presented with two choices: either to create a user account or to create a company account. The user account creation process leads directly to access, while the company account requires verification by the main administrator before the account becomes active. Once either type of account is successfully created and approved, the user is directed to the application's homepage.

On the other hand, if the account already exists, users are given multiple options: they can choose to log in as a regular user or a company, or they may initiate the “Forgot Password” process in case they are unable to access their account. The password recovery process includes entering the registered email and receiving a reset link, which once completed, also leads the user to the homepage.

Upon successful login or account recovery, users access the homepage, which acts as a central navigation point for the application. From there, users can manage their personal information by accessing the “View Profile” option. If necessary, they can update their profile, and the updated information is saved, completing the loop back to the view profile page or concluding the session.

The diagram provides a clear and structured visual representation of the user flow for both standard users and company administrators, ensuring a smooth onboarding and management experience in the SAFAR app.

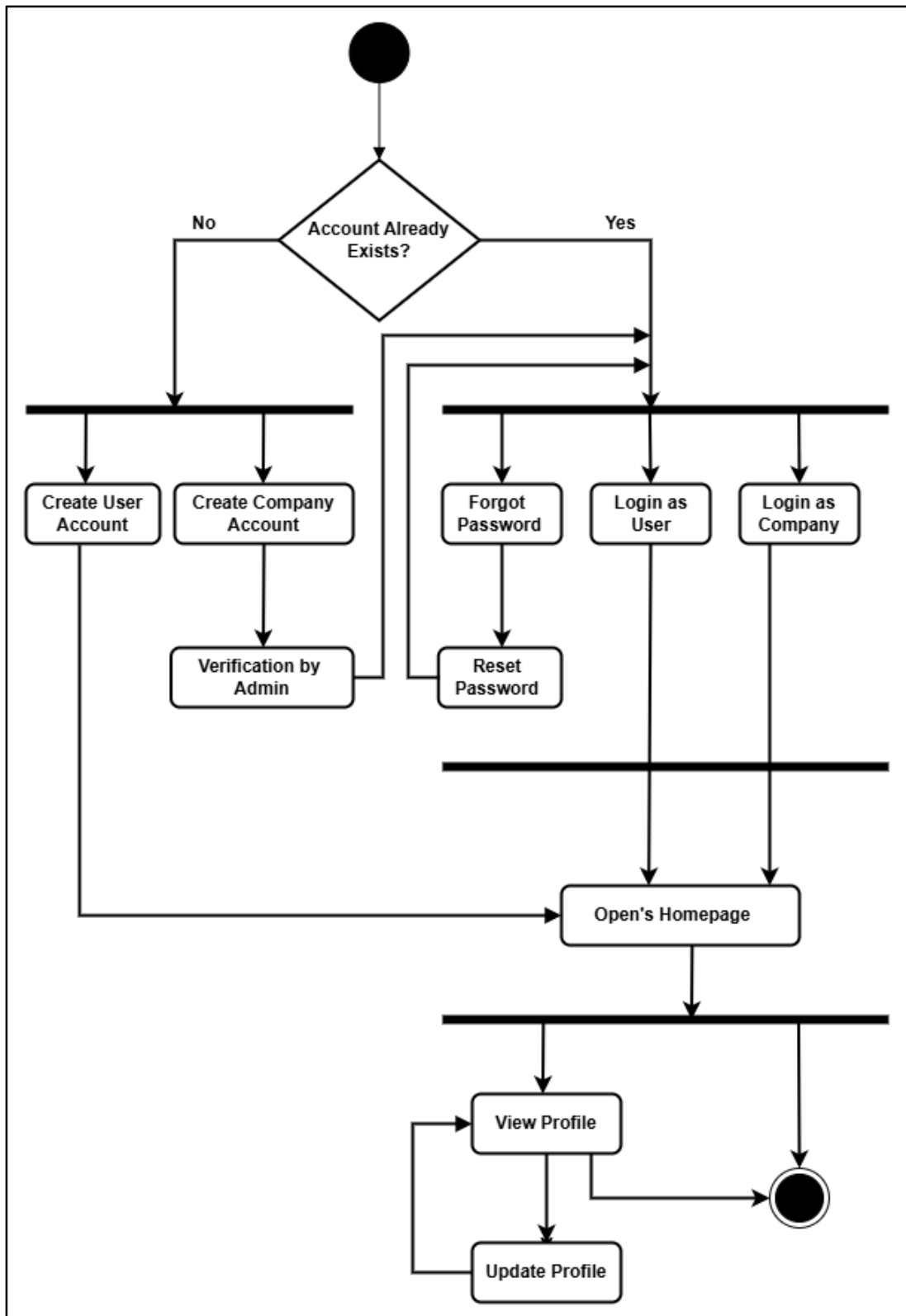


Figure 4.2: Activity Diagram of Authentication

This is an **activity diagram** that visually represents the flow of activities within a system, possibly for an administrative user managing an application. The process begins at the **initial node**, represented by the solid black circle at the top.

The first activity is **Login**, where the user is required to authenticate themselves to access the system. After a successful login, the user is presented with two main options or parallel paths, represented by the horizontal fork:

1. Approve Companies:

In this path, the user can review and approve company accounts or registrations. Once the approval process is completed, the next activity is **Notify Users**.

2. User's Account Information:

Alternatively, the user can choose to access **User's Account Information**. This activity likely allows the user to view, manage, or update details related to users within the system (e.g., their profiles, status, permissions, etc.).

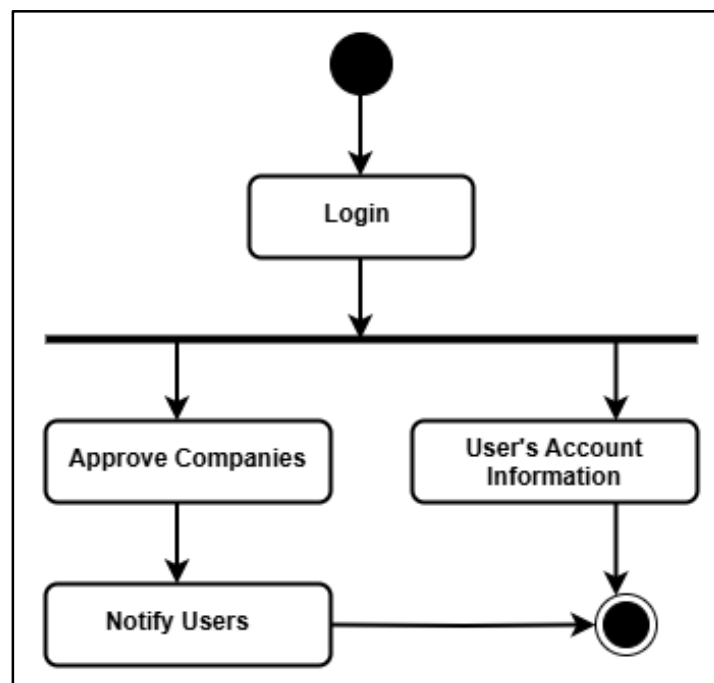


Figure 4.3: Activity Diagram of Admin

This activity diagram represents the workflow of a Company Dashboard system, typically used by transportation or bus service companies. The process starts at the Login stage, followed by access to the Company Dashboard. From the dashboard, users can perform multiple management activities in parallel. These include: Managing Drivers, Adding/Updating Payment Details, Managing Buses, Managing Profile, and Managing Tickets. If the user chooses to update payment details, they proceed to Schedules, where a decision is made: if the routes already exist, the user can directly Add Schedules; if not, they must first Add Routes, then add schedules. Meanwhile, under Manage Tickets, the user can either Approve Tickets—leading to a Notify User action—or Cancel Tickets, which triggers a Refund Payment.

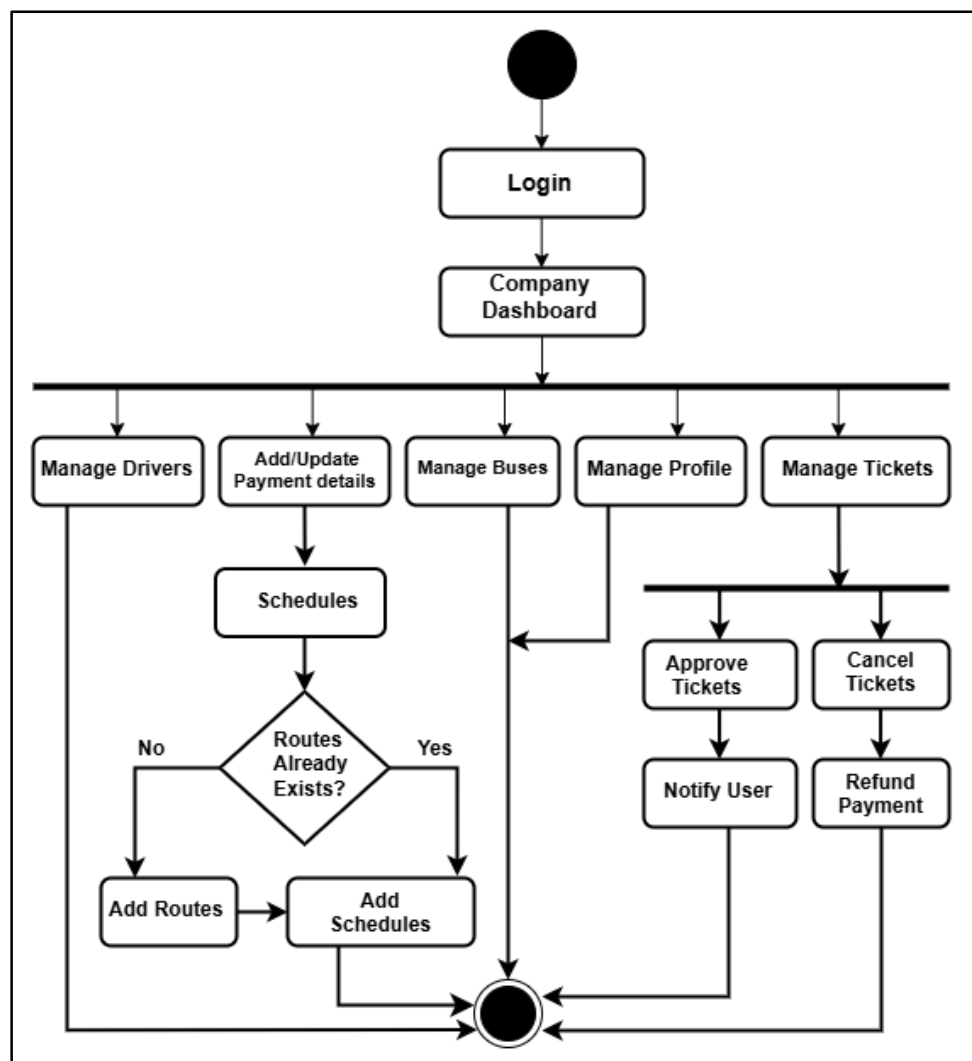


Figure 4.4: Activity Diagram of Company Admin

This activity diagram illustrates the complete workflow of a bus ticket booking application from the user's perspective.

The process begins with the user logging into the system, which then directs them to the HomeScreen, the central hub for all navigation. From the HomeScreen, users can access three main paths: the Drawer menu, the ticket booking flow, and the booked tickets section. The Drawer menu includes functionalities for Profile Management and accessing Contact Options and Policies, allowing users to manage their account details and review important service guidelines.

The core booking flow starts when the user selects their desired route and date. They then choose a company and seat type, after which the system displays the available tickets. Users can apply filters to narrow down seat selection based on personal preferences or requirements. Once a suitable seat is chosen, they proceed to view ticket details, ensuring all information is correct before making a purchase.

Finally, the user completes the process by sending the payment, which marks the successful booking of the ticket.

In parallel, the system allows users to manage previously **booked tickets**. From this section, they can decide whether they want to **cancel a ticket**. If not, they simply wait for the trip as planned.

However, if they do want to cancel, the system asks whether they want to **apply for a refund**. If yes, a refund request is initiated; if not, they may need to **wait for ticket approval**, possibly from the service provider.

Regardless of the path taken, all user actions eventually return to a unified end point, ensuring a closed and well-monitored system loop.

Overall, the diagram outlines a clear, user-friendly interface and logical flow of actions for users to follow within the bus booking app.

It provides a seamless experience from login to booking management, while offering additional options for profile configuration and service support.

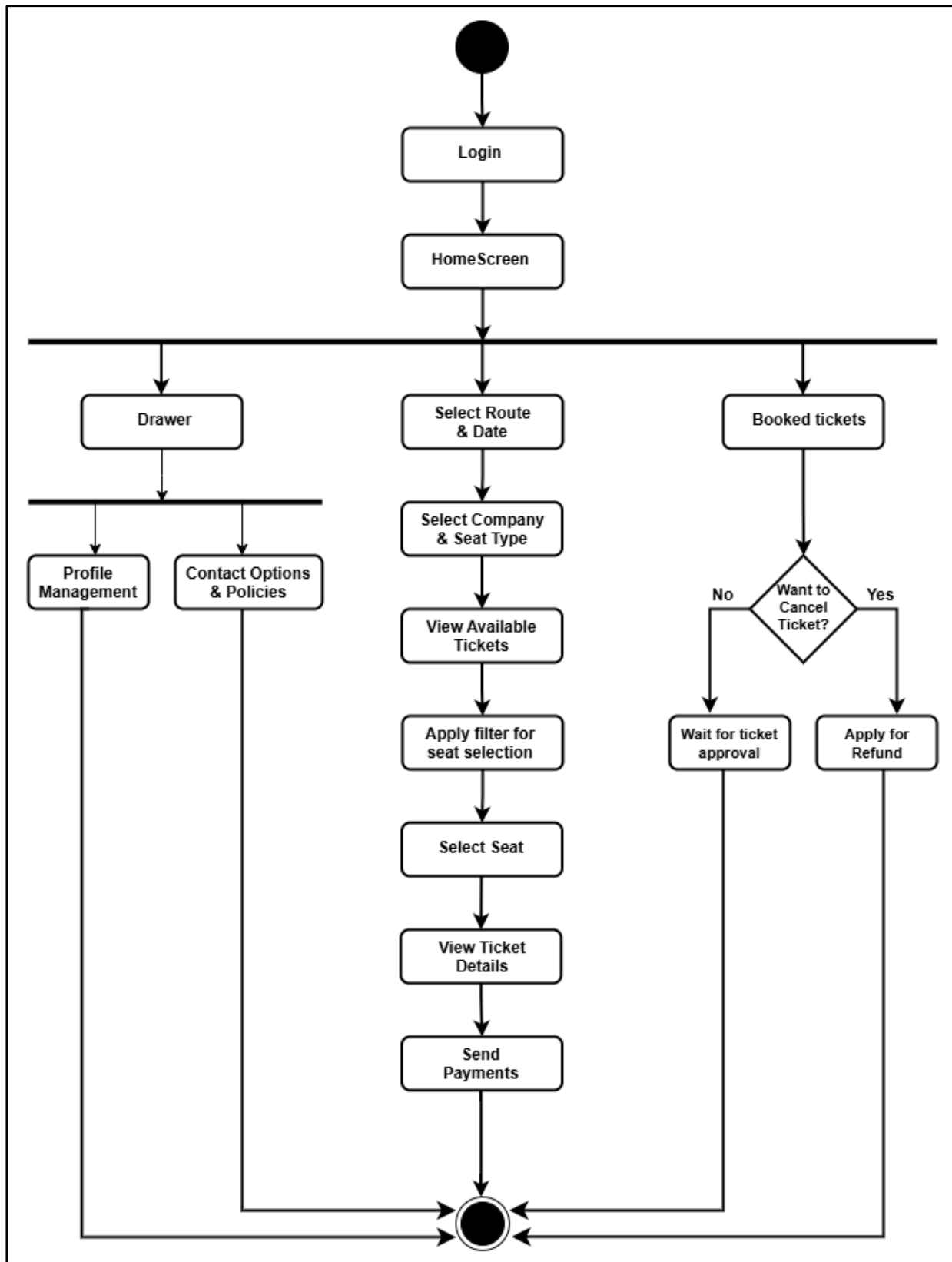


Figure 4.5: Activity Diagram of User

4.3 Data Definition Diagram

The figure 4.6 represents the class diagram of Safar in which classes like user, seat selection, payment, feedback, notification, ticket, company admin, schedule etc are shown.

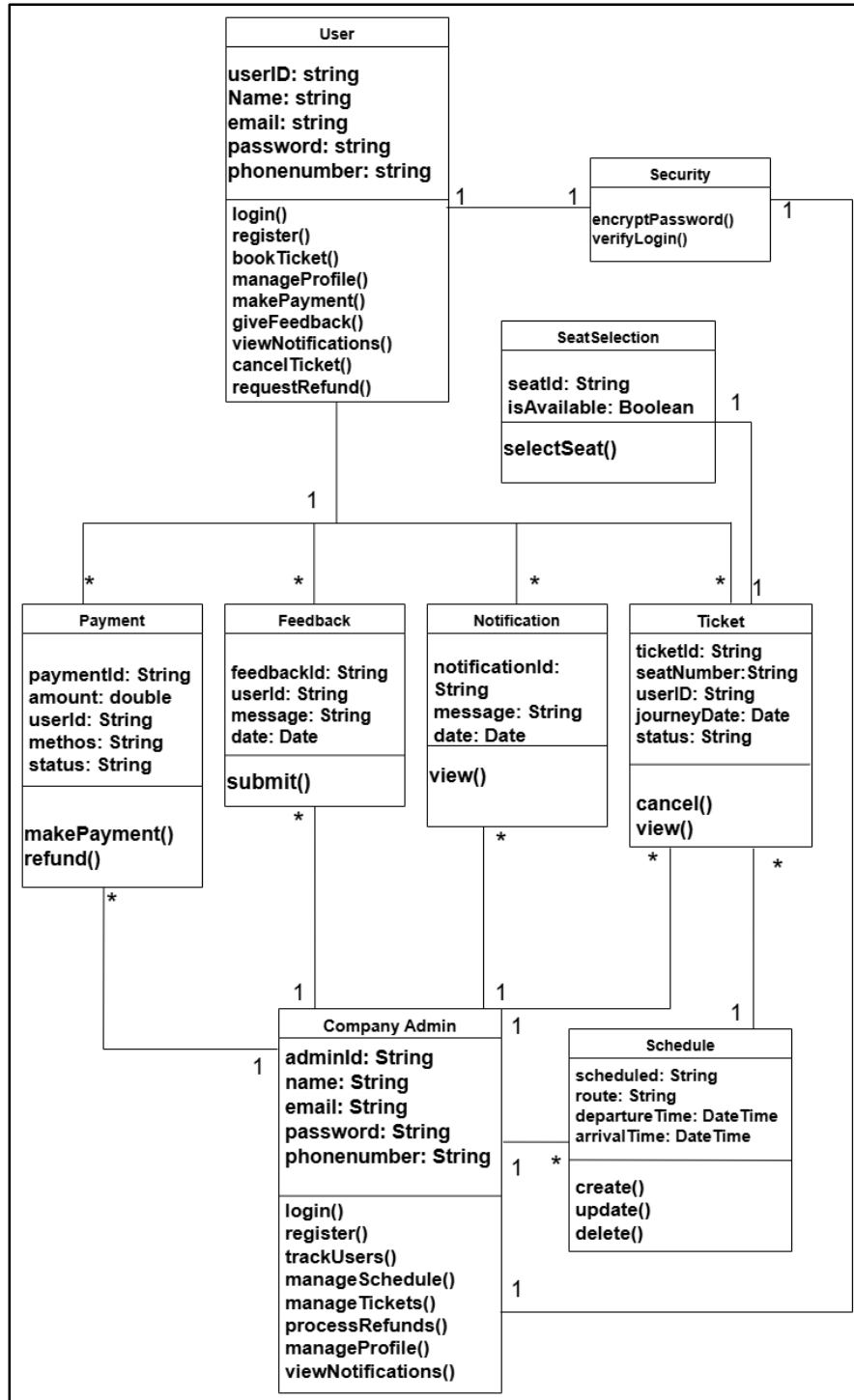


Figure 4.6: Class Diagram of Safar

4.4 Sequence Diagram

The figure 4.7, 4.8 and 4.9 shows the sequence diagram of Safar; it shows the functionalities of the modules like login, ticket booking, managing company admin side, user and admin side.

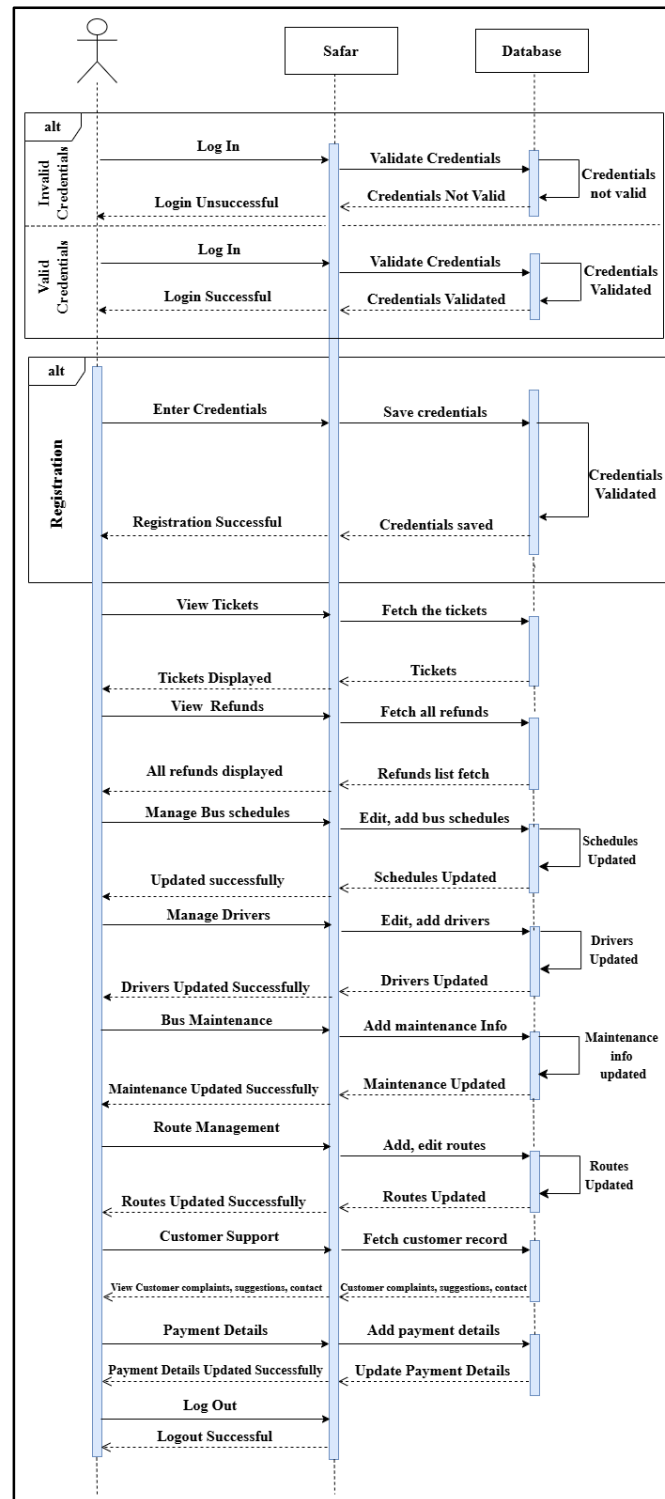


Figure 4.7: Sequence Diagram of Company Admin

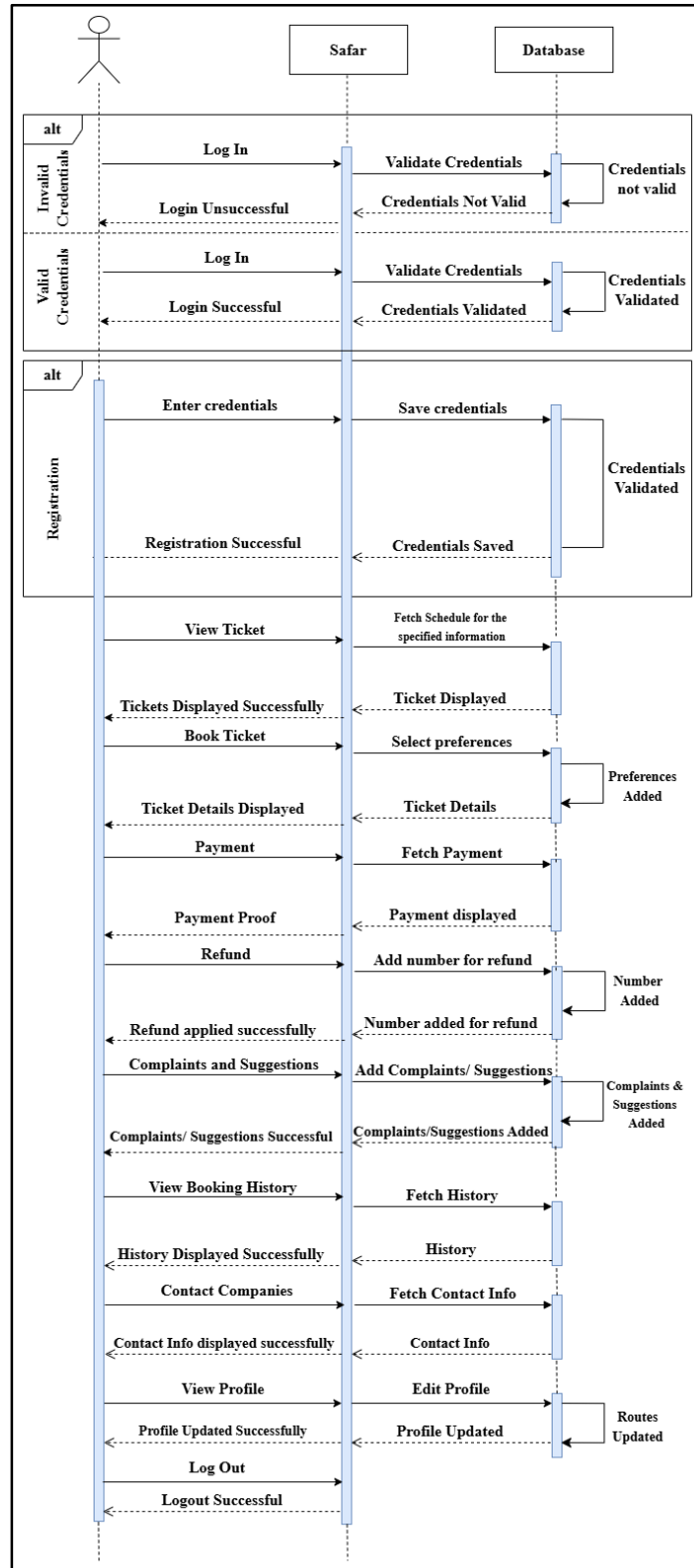


Figure 4.8: Sequence Diagram of User

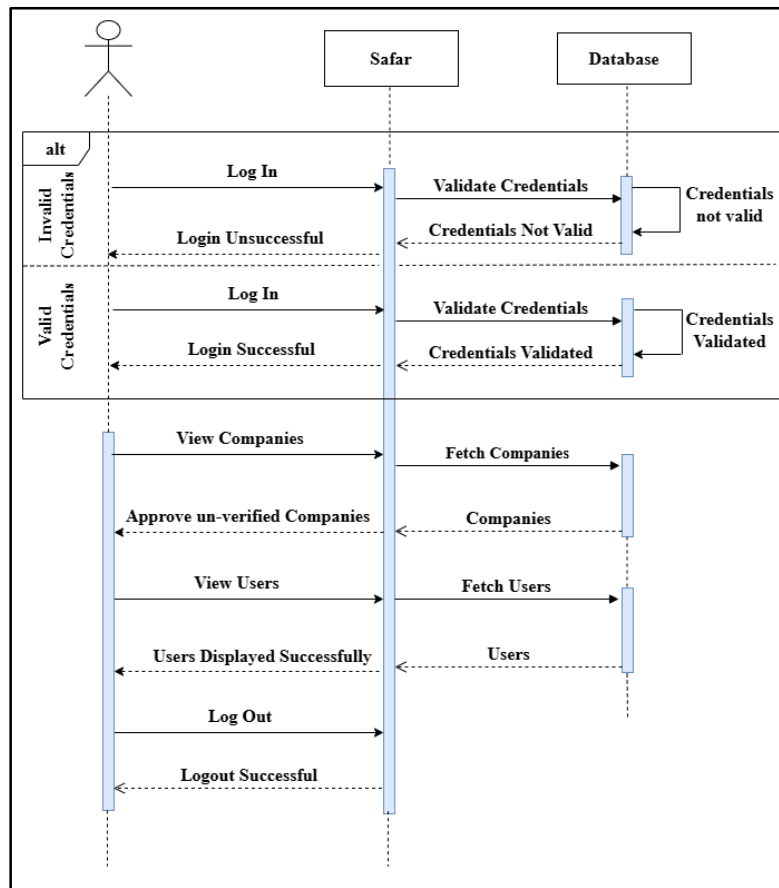


Figure 4.9: Sequence Diagram of Admin

4.5 State Transition Diagram

The following are the state transition diagrams of our system:

- **Opening:** The process begins, likely indicating the start of a ticket booking or payment submission system.
- **Submit Payment:** The user submits payment details for the ticket.
- **Submission:** The payment details are processed or confirmed.
- **Upload Image:** The user is prompted to upload an image (possibly for verification, such as an ID or payment proof).
- **Uploading:** The system is in the process of receiving the image.
- **Analyze Image:** The uploaded image is being analyzed (e.g., for authenticity or compliance).
- **Under Analysis:** The system continues to review the image.

- **payment = correct:** A confirmation that the payment details are verified and correct.
- **Ticket Booked:** The ticket is successfully booked.
- **Approved by admin:** Final approval is granted by an administrator, completing the process.

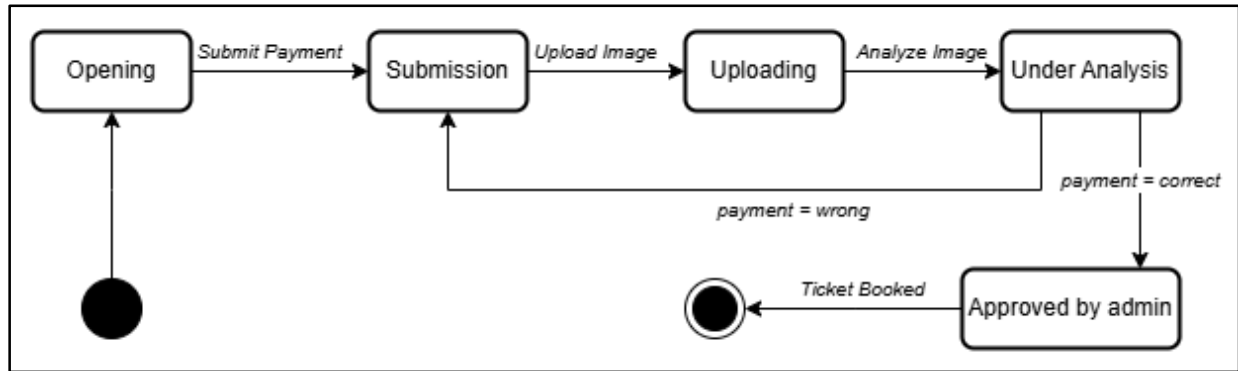


Figure 4. 10: State Diagram of Payment Verification

This state diagram outlines the step-by-step ticket cancellation.

- **Opening:** The process begins, indicating the initiation of a cancellation request.
- **Cancellation:** The user selects or confirms the intent to cancel a booking or service.
- **Selection Payment Method:** The system prompts the user to choose a preferred payment method for the refund.
- **Payment Method Selected:** The user's selection is confirmed.
- **Entering Refund Details:** The user provides necessary details for processing the refund (e.g., account information or reason for cancellation).
- **Click Apply:** The user submits the refund request.
- **Applied:** The system confirms that the refund request has been successfully processed or initiated.

This activity diagram represents a **user interface flow or backend process** for handling cancellations, emphasizing payment method selection and refund details submission. The final "Applied" step suggests completion, though additional steps (e.g., approval or processing time) may not be shown.

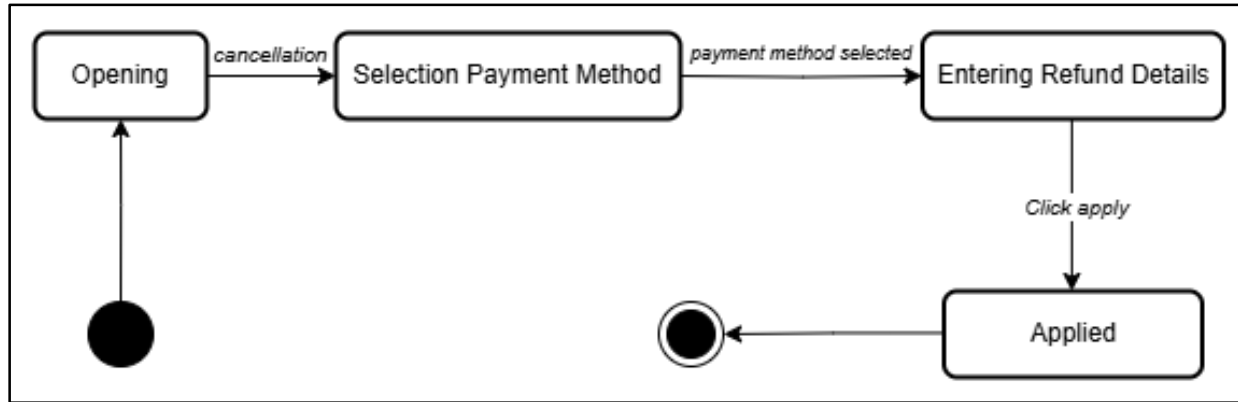


Figure 4. 11: State Diagram of Refund Process

4.6 Data Design

The following are the data design of Safar in tabular form:

4.6.1 Data Dictionary

The following is the data dictionary of our system:

Table 4.1: Data Collection of User Model

Field Name	Data Type	Constraint	Description
userName	String	required, non-nullable	Name of the user
uid	String	required, non-nullable (primary key)	Unique ID of the user
phoneNumber	String	required, non-nullable	User's contact number
imageUrl	String	required, non-nullable	URL of the user's profile image
isUser	bool	required, non-nullable	Indicating if the account is a regular user
isVerified	bool	required, non-nullable	Showing if the account has been verified.
isAdmin	bool	required, non-nullable	Indicates if the user has admin privileges.
gender	String	required, non-nullable	Gender of the user
password	String?	Optional, nullable	User password

fcmToken	String	required, non-nullable	Firebase Cloud Messaging is taken for notifications.
companyName	String	required, non-nullable	Company name
numberOfBus	int	required, non-nullable	Total buses associated with the company.
email	String	required, non-nullable	User email address

Table 4.2: Data Collection of Booking Model

Field Name	Data Type	Constraints	Description
id	String	required, non-nullable	Booking record ID
companyId	String	required, non-nullable (foreign key)	ID of the bus company
scheduleId	String	required, non-nullable (foreign key)	ID of the bus schedule
passengerName	String	required, non-nullable	Name of the passenger
arrival	String	required, non-nullable	Arrival location name
paymentProof	String	required, non-nullable	Proof of payment
arrivalTerminal	String	required, non-nullable	Arrival terminal name
departureTerminal	String	required, non-nullable	Departure terminal name
isFamily	Boolean	required, non-nullable	Indicates if booking is for a family
isMale	Boolean	required, non-nullable	Passenger gender (true = male)
arrivalTime	Timestamp/DateTime	required, non-nullable	Arrival date and time
bookingNumber	String	required, non-nullable	Unique booking number
status	String	required, non-nullable	Current booking status
isBussinessClass	Boolean	required, non-nullable	Indicates business class booking

seatNums	Array<Integer>	required, non-nullable	List of seat numbers
departure	String	required, non-nullable	Departure location name
totalPrice	String	required, non-nullable	Final price of the booking
departureDate	Timestamp/DateTime	required, non-nullable	Date of departure
departureTime	Timestamp/DateTime	required, non-nullable	Departure time
bookingTime	Timestamp/DateTime	required, non-nullable	Booking Time
prices	String	required, non-nullable	Price per seat or ticket
contact	String	required, non-nullable	Passenger's contact information
userId	String	required, non-nullable (foreign key)	ID of the user who made the booking

Table 4.3: Data Collection of Bus Schedule

Field Name	Data Type	Constraints	Description
id	String	required, non-nullable	Unique identifier of the schedule
busNumber	String	required, non-nullable	Number of the bus
userId	String	required, non-nullable (foreign key)	ID of the company who created it
deparureTerminal	String	required, non-nullable	Starting terminal of the journey
arrivalTerminal	String	required, non-nullable	Destination terminal of the journey
provideServices	List<int>	required, non-nullable	IDs of services provided
departure	String	required, non-nullable	Departure city
arrival	String	required, non-nullable	Arrival city
departureTime	DateTime	required, non-nullable	Time of departure
departureDate	DateTime	required, non-nullable	Date of departure

arrivalTime	DateTime	required, non-nullable	Time of arrival
noOfSeat	int	required, non-nullable	Total number of seats
createdAt	DateTime	required, non-nullable	Record creation time
bookedSeats	List<BookedSeat>	required, non-nullable	List of booked seats
status	bool	required, non-nullable	Is the schedule active or inactive
prices	String	required, non-nullable	Ticket price or pricing details
isBussinessClass	bool	required, non-nullable	Whether it's a business class bus

Table 4.4: Data Collection of Bus Maintenance Model

Field Name	Data Type	Constraints	Description
id	String	required, non-nullable	Identifier for the maintenance record
busNumber	String	required, non-nullable	Bus no associated with record
lastService	DateTime	required, non-nullable	Date when the bus was last serviced
nextService	DateTime	required, non-nullable	Scheduled date for the next service
status	bool	required, non-nullable	Indicates whether the bus is in working condition
createdAt	DateTime	required, non-nullable	Timestamp when the maintenance record was created
userId	String	required, non-nullable (foreign key)	ID of the user who created the record

Table 4.5: Data Collection of Chat Model

Field Name	Data Type	Constraints	Description
id	String	required, non-nullable	Unique identifier of the message
message	String	required, non-nullable	The chat message content

messageTime	DateTime	required, non-nullable	Time when the message was sent
senderId	String	required, non-nullable (foreign key)	ID of the sender of the message
threadId	String	required, non-nullable (foreign key)	ID of the conversation thread

Table 4.6: Data Collection of Complaint & Suggestion Model

Field Name	Data Type	Constraint	Description
id	String	required, non-nullable	Unique identifier
companyId	String	required, non-nullable, fk	Company receiving the message
senderId	String	required, non-nullable, fk	The user who sent the message
text	String	required, non-nullable	The content of the complaint/suggestion
isComplaint	bool	required, non-nullable	True if it's a complaint, false if suggestion
createdAt	DateTime	required, non-nullable	Date and time when it was created

Table 4. 7: Data Collection of Drivers Model

Field Name	Data Type	Constraint	Description
driverNumber	String	required, non-nullable	Unique identifier for the driver
id	String	required, non-nullable	Unique ID
name	String	required, non-nullable	Full name of the driver
status	bool	required, non-nullable	Driver's active/inactive status
phoneNumber	String	required, non-nullable	Contact number of the driver
userId	String	required, non-nullable (foreign key)	Refers to company who added driver
createdAt	DateTime	required, non-nullable	Date and time when it was created

Table 4.8: Data Collection of Notifications Model

Field Name	Data Type	Constraint	Description
id	String	required, non-nullable	Unique ID of the notification
title	String	required, non-nullable	Title of the notification
body	String	required, non-nullable	Message body of the notification
profileImage	String	required, non-nullable	URL of sender's profile image
topic	String	required, non-nullable	Topic/channel of the notification
receiverIds	List<String>	required, non-nullable	Users who receive notification
senderId	String	required, non-nullable, (FK)	ID of the sender
timestamp	DateTime	required, non-nullable	When the notification was sent

Table 4.9: Data Collection of Payment Model

Field Name	Data Type	Constraint	Description
companyId	String	required, non-nullable (foreign key)	ID of the company
id	String	required, non-nullable	ID for the payment contact record
jazzCashNumber	String	required, non-nullable	Company's JazzCash number
easyPaisaNumber	String	required, non-nullable	Company's EasyPaisa number

Table 4.10: Data Collection of Refund Model

Field Name	Data Type	Constraint	Description
id	String	required, non-nullable	Unique identifier for the refund
userId	String	required, non-nullable (foreign key)	ID of the user who made the refund
companyId	String	required, non-nullable (foreign key)	ID of the company handling the refund
paymentMethod	String	required, non-nullable	The method used for the payment

name	String	required, non-nullable	User associated with the refund
paymentProof	String	required, non-nullable	A proof document of the payment
ticketNumber	String	required, non-nullable	The ticket number related to the refund
accountNumber	String	required, non-nullable	Account number used for refund
status	String	required, non-nullable	The status of the refund
createdAt	DateTime	required, non-nullable	Date and time when refund was created.
company	UserModel?	nullable	The associated company details
amount	String	required, non-nullable	The amount refunded

Table 4. 11: Data Collection of Route Model

Field Name	Data Type	Constraint	Description
id	String	required, non-nullable	Unique identifier for the route
routeId	String	required, non-nullable	The ID of the route
userId	String	required, non-nullable	User who created the route
departure	String	required, non-nullable	Departure location of the route
arrival	String	required, non-nullable	Arrival location of the route
distance	String	required, non-nullable	Distance of the route
createdAt	DateTime	required, non-nullable	Date and time when the route was created
status	bool	required, non-nullable	The status of the route

Chapter 5

Implementation

5. Implementation

5.1 Algorithm

The following are module wise algorithms for Safar:

5.1.1 User Profiling

1. Signup as User

Algorithm 1: Signup as user
Input: Username, email, password, phone number
Output: Response
Function signupUser (): # Step 1: Ask user to enter Input # Step 2: Save User Record in the database

2. Signup as Company

Algorithm 1: Signup as company
Input: Companyname, email, password, number of buses
Output: Response
Function signupCompany (): # Step 1: Ask company admin to enter details # Step 2: Save Company info temporarily in database with status = 'pending' # Step 3: Notify Main_Admin for verification # Step 4: Display message ("Company registration request submitted")

3. Login

Algorithm 1: Login
Input: email, password
Output: Response
Function loginProcess (): # Step 1: Check credentials # Step 2: Validate credentials

4. Company Verification by Admin

Algorithm 1: Main Admin Verification
Input: email, password
Output: Response
Function adminVerification (): # Step 1: Check unverified companies # Step 2: Update company status to 'verified' # Step 3: Notify company # Step 4: Display message ("Company verified successfully")

5.1.2 Company Admin

1. Add Payment Numbers

Algorithm 1: Add payment numbers
Input: JazzCash and EasyPaisa numbers
Output: Response

Function paymentNumbers ():

Step 1: Get company ID from session

Step 2: Store JazzCash and EasyPaisa numbers in database

Step 3: Display message ("Payment numbers added successfully")

2. Add Route and Schedule

Algorithm 1: Add route and schedule

Input: routeInfo, scheduleDetails

Output: Response

Function addRouteSchedule ():

Step 1: Enter route info

Step 2: Enter schedule details

Step 3: Save details in database

Step 4: Display schedule in Bus schedule screen

3. Approve Booking

Algorithm 1: Approve Booking

Input: ticketID, TID, screenshot

Output: Response

Function approveBooking ():

Step 1: Verify payment using TID

Step 2: Update ticket status to 'confirmed'

Step 3: Notify user

Step 4: Display message ("Ticket approved")

4. Refund Process

Algorithm 1: Refund Process

Input: refundRequestID, transactionScreenshotOutput

Output: Response

Function refundProcess ():

Step 1: Verify request and screenshot

Step 2: Send payment to user's number

Step 3: Notify user

Step 4: Display message ("Refund successful")

5.1.3 User Side

1. Select Route and filter

Algorithm 1: Select Route and Filter

Input: departureCity, arrivalCity, travelDate

Output: filteredCompanies

Function filterRoutes ():

Step 1: Get user input (departure, arrival, date)

Step 2: Display filter screen with options (Company, Class)

Step 3: Return filtered results

2. Select Seats

Algorithm 1: Select seats

Input: selectedTicket, numberOfSeats, gender

Output: Response

Function selectSeats ():

Step 1: Show seat layout (color-coded)

Step 2: Allow seat selection based on gender logic

Step 3: If mismatch (e.g., male selects female seat), show warning

Step 4: Select seat and display confirmation ticket

3. Book Ticket and Payment

Algorithm 1: Book Ticket and payment

Input: selectedSeats, userDetails, paymentMethod, screenshot

Output: Response

Function bookTicket ():

Step 1: Display payment instructions with company payment numbers

Step 2: Upload screenshot and store TID

Step 3: Save ticket info with status = 'pending'

Step 4: Display message ("Booking submitted. Approval by admin in 45 mins")

4. Ticket Cancellation and Refund Request

Algorithm 1: Ticket Cancellation and Refund Request

Input: bookingID

Output: Response

Function cancelTicket ():

Step 1: Click on cancel ticket button

Step 2: Display Refund screen and process

Step 3: Submit refund request to company # Step 4: Display message ("Refund requested")
--

5. Profile Management

Algorithm 1: Profile Management
Input: userID, updatedDetails
Output: Response
Function updateProfile (): # Step 1: Get user ID from session # Step 2: Allow users to update photos, phone numbers and password # Step 3: Save changes to database # Step 4: Display message ("Profile updated")

6. Complaint & Suggestions

Algorithm 1: Complaint & Suggestions
Input: userID, message
Output: Response
Function sendFeedback (): # Step 1: Select Company to send complaint & suggestions # Step 2: Type message and click on Send button # Step 3: Store in suggestions/complaints database # Step 4: Notify company admin # Step 4: Display message ("Feedback submitted")

7. Chatbot Contact

Algorithm 1: Chatbot Contact
Input: userQuery
Output: Response
Function chatWithBot (): # Step 1: Select Company to contact # Step 2: Display chat with company screen # Step 3: Send message # Step 4: Notify company admin

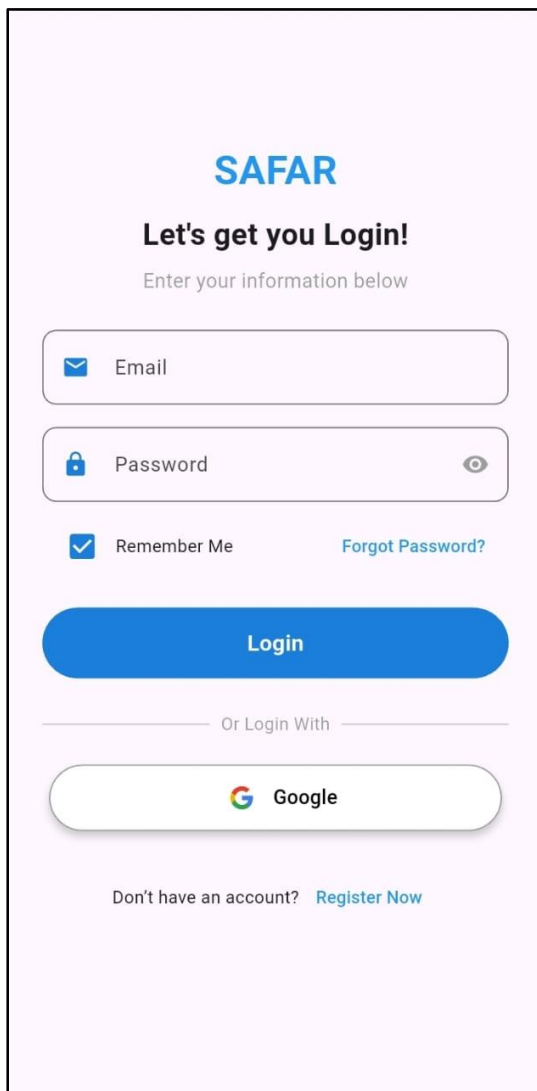
5.2 External APIs

Table 5.1: External APIs used in Safar

Name & Version	Description	Purpose of Usage	API Endpoint Function in which it is used
Firebase (latest)	Backend as a Service platform from Google	User authentication, password reset, and real-time database	firebase.auth().signInWithEmailAndPassword() firebase.auth().sendPasswordResetEmail()
Supabase (latest)	Open-source Firebase alternative with PostgreSQL and storage	Image and file storage (e.g. payment screenshots, update profile images), database	supabase.storage.from('screenshots').upload()

EmailJS (version 4.4.1)	Email API to send emails directly from the frontend	Sending email for password reset and booking confirmations	emailjs.send()
--	---	--	----------------

5.3 User Interface



SAFAR

Let's get you Login!

Enter your information below

☒ Remember Me [Forgot Password?](#)

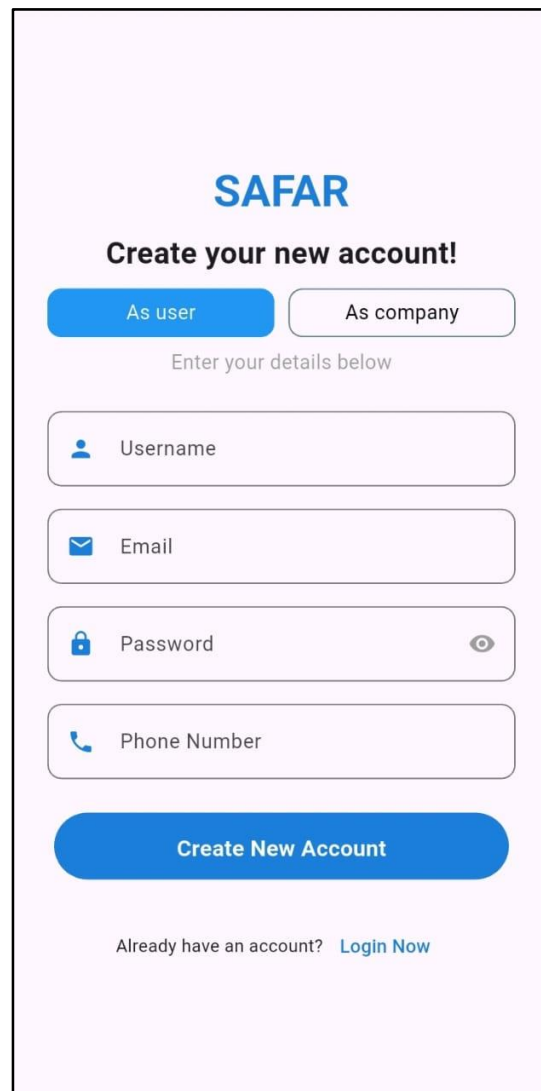
Login

Or Login With

 Google

Don't have an account? [Register Now](#)

Figure 5.1: Login UI



SAFAR

Create your new account!

As user **As company**

Enter your details below

Create New Account

Already have an account? [Login Now](#)

Figure 5.2: Signup UI

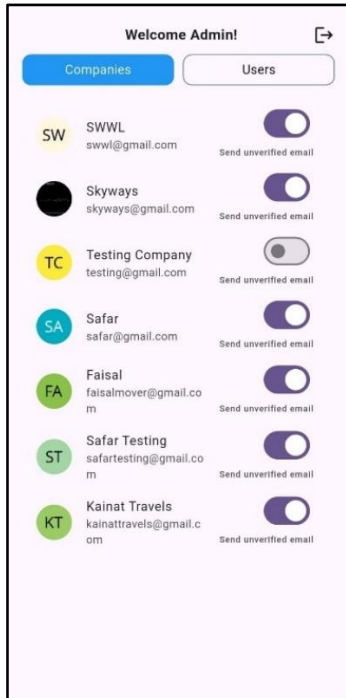


Figure 5.3: Companies Verification UI

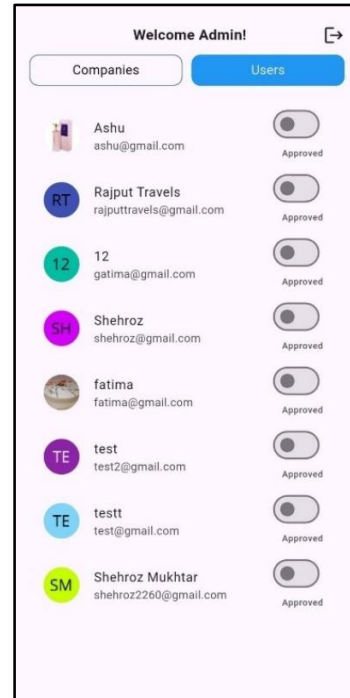


Figure 5.4: Users Verification UI

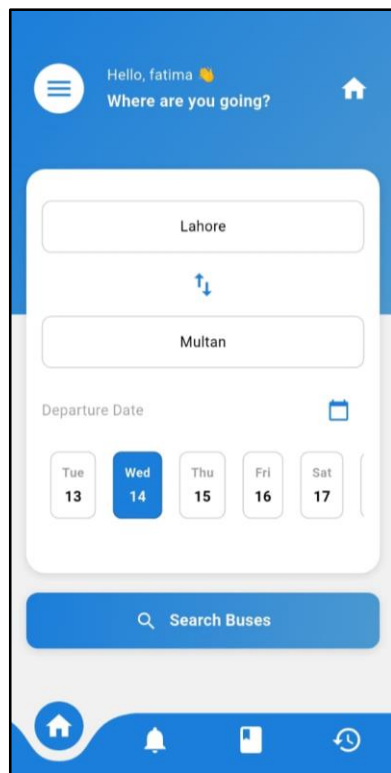


Figure 5.5: Homepage UI

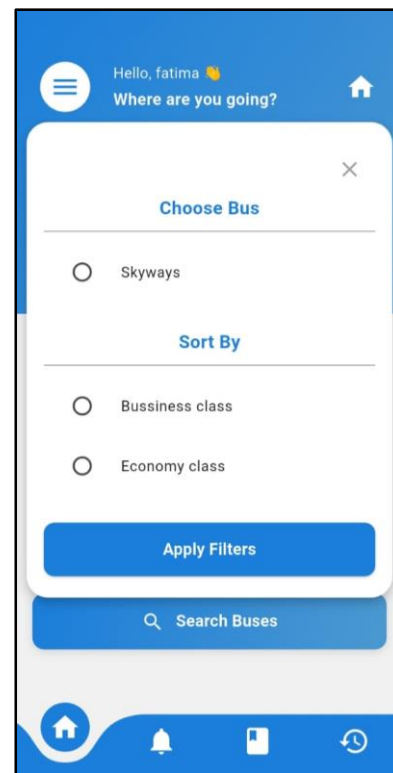


Figure 5.6: Filter UI

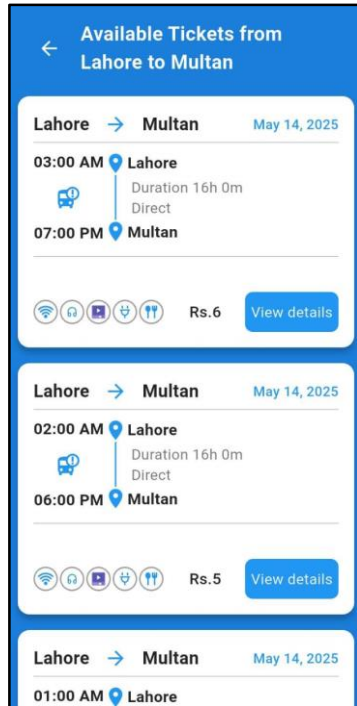


Figure 5.7: Available Tickets UI

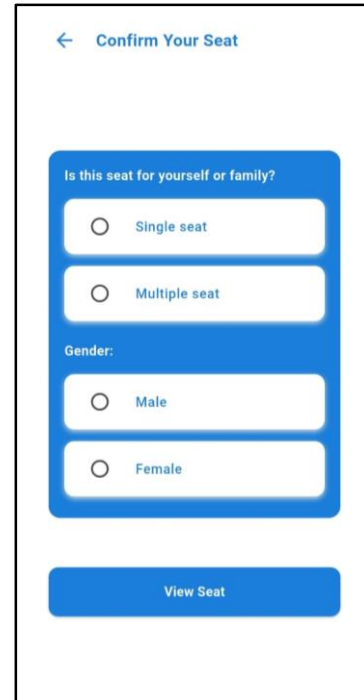


Figure 5.8: Choose Yourself UI

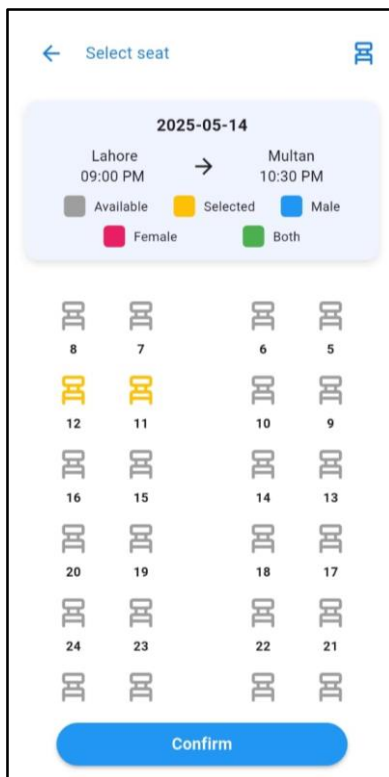


Figure 5.9: Bus Seat UI

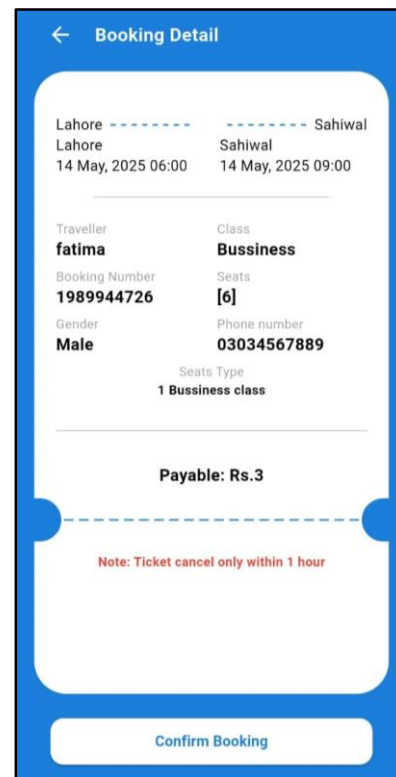


Figure 5.10: Ticket Detail UI

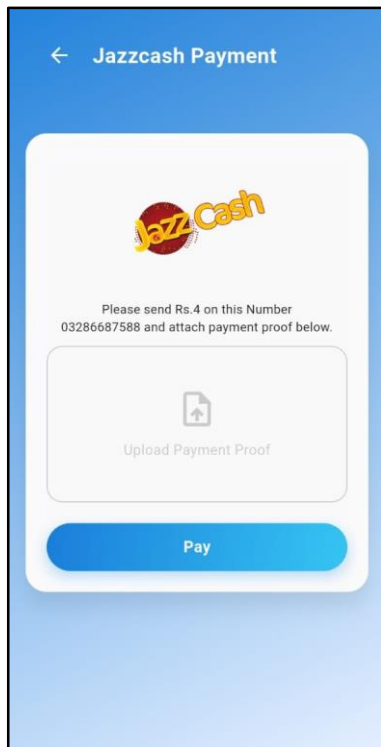


Figure 5.11: JazzCash UI

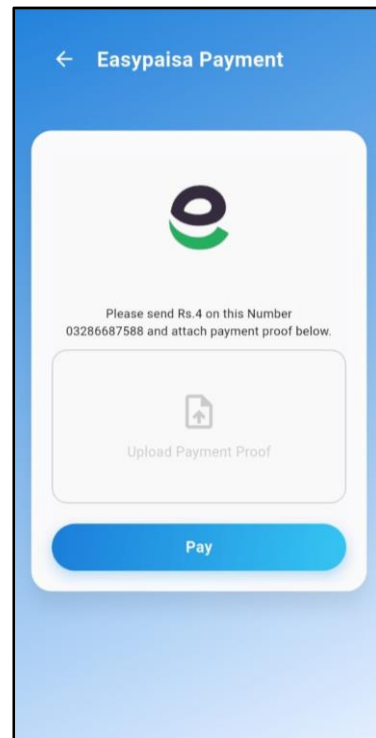


Figure 5.12: EasyPaisha UI

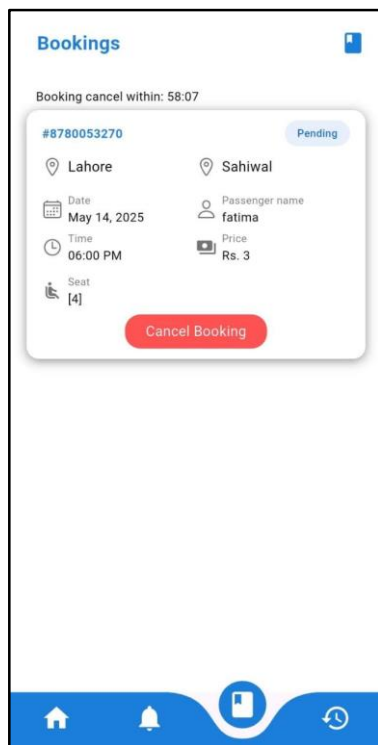


Figure 5.13: Booked Ticket UI

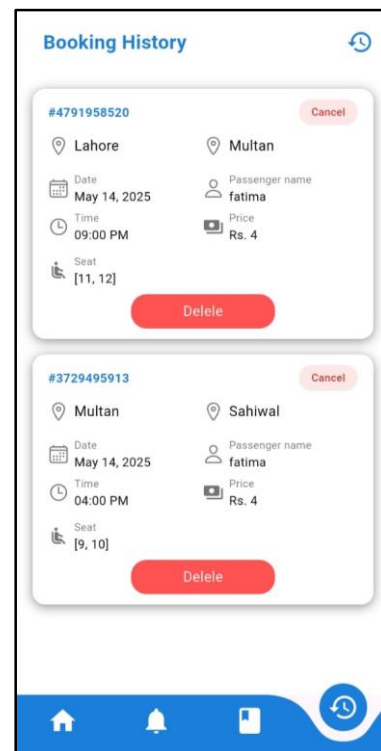


Figure 5.14: Booking History UI

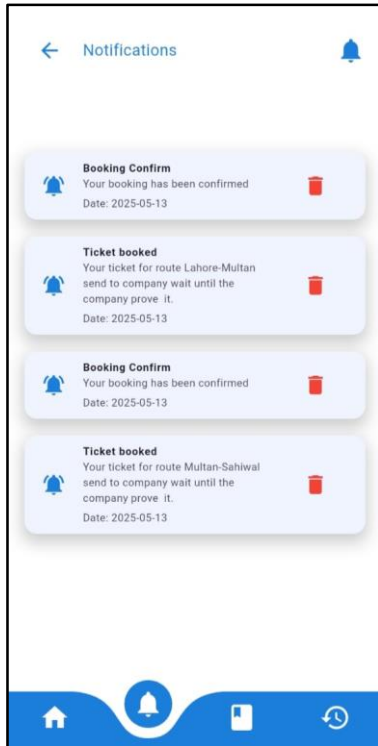


Figure 5.15: Notifications UI

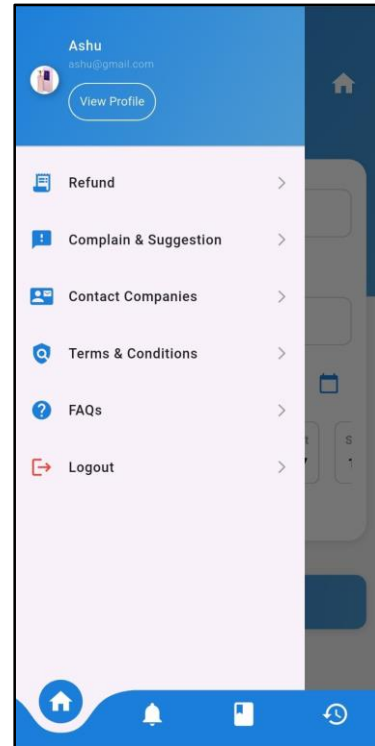


Figure 5.16: Drawer UI

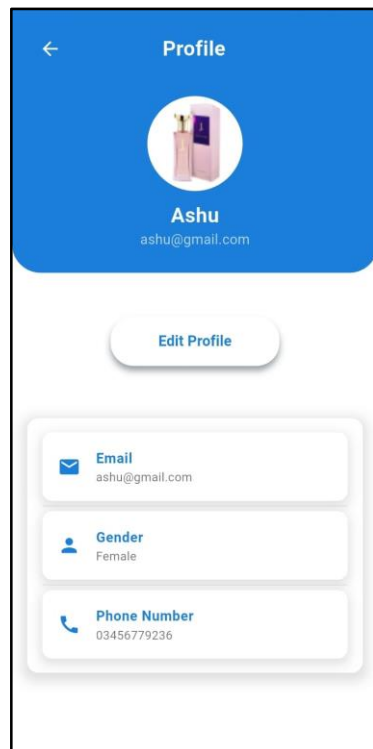


Figure 5.17: User Profile UI

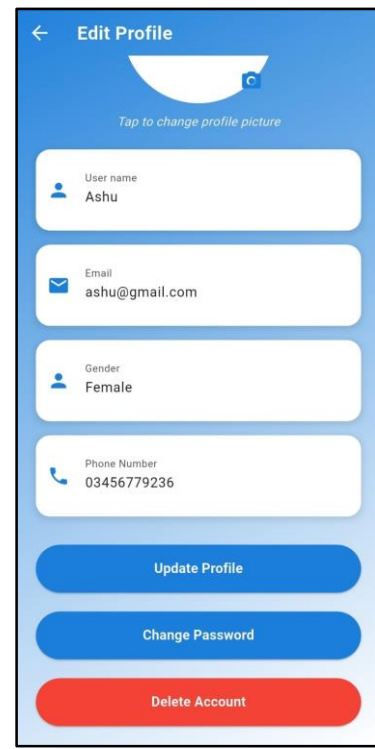


Figure 5.18: Edit Profile UI

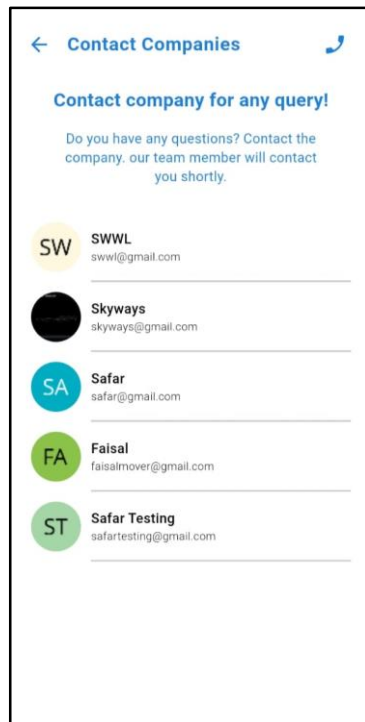


Figure 5.19: Contact Companies UI

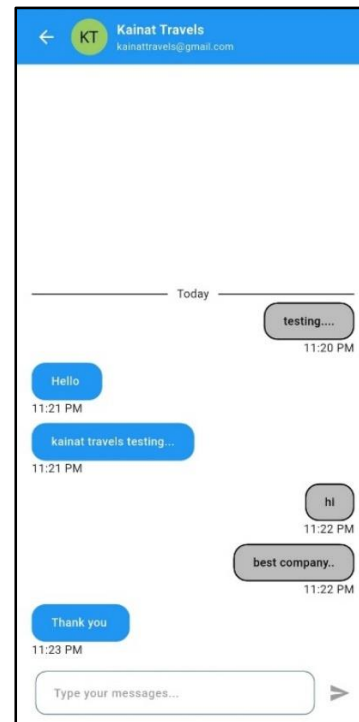


Figure 5.20: Contact Chatbot UI

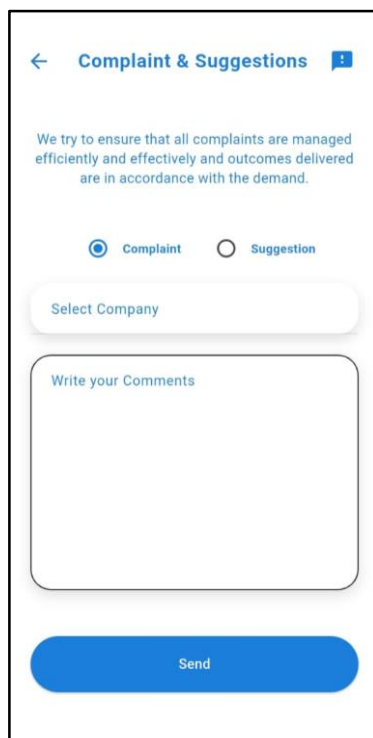


Figure 5.21: Complaint & Suggestion UI

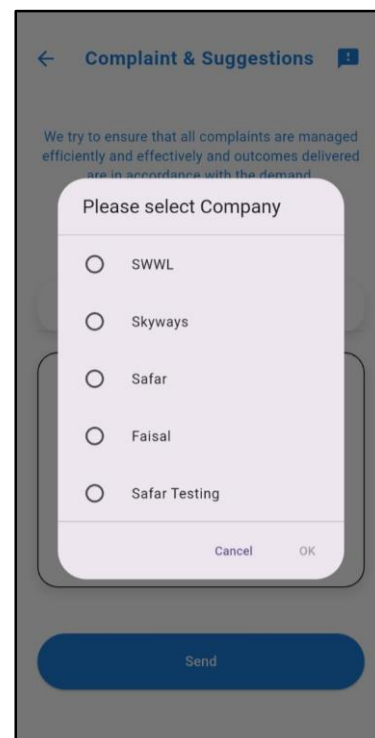


Figure 5.22: Company Selection UI

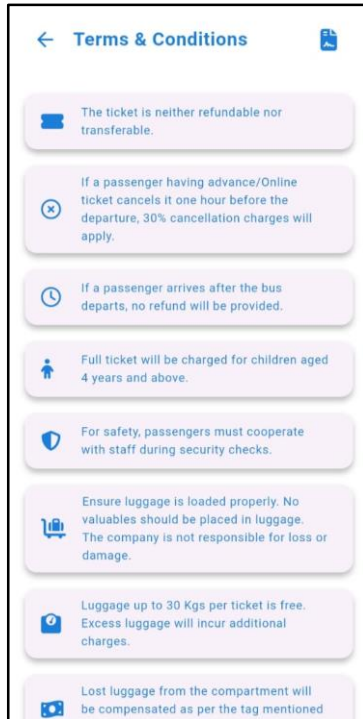


Figure 5.23: Terms & Conditions UI

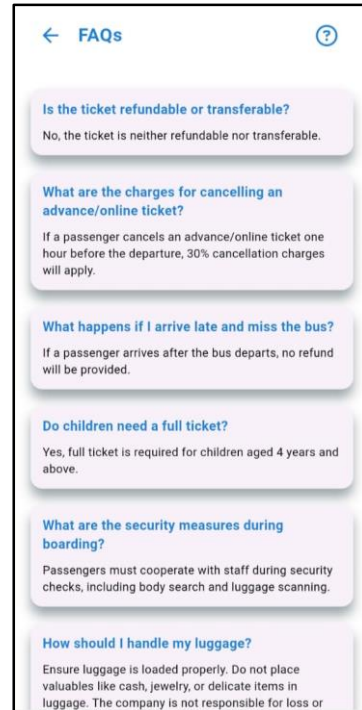


Figure 5.24: FAQs UI

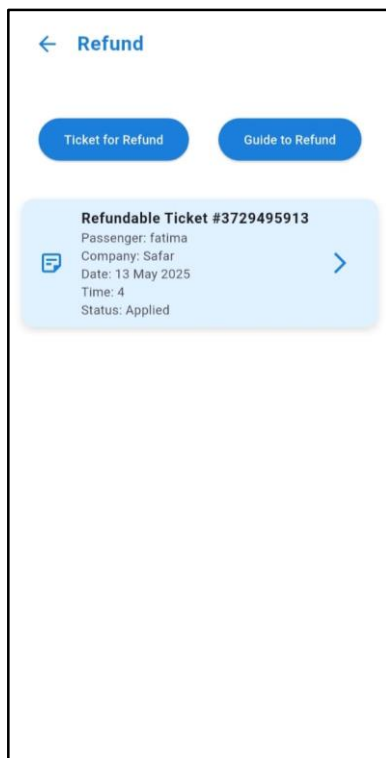


Figure 5.25: Refund UI

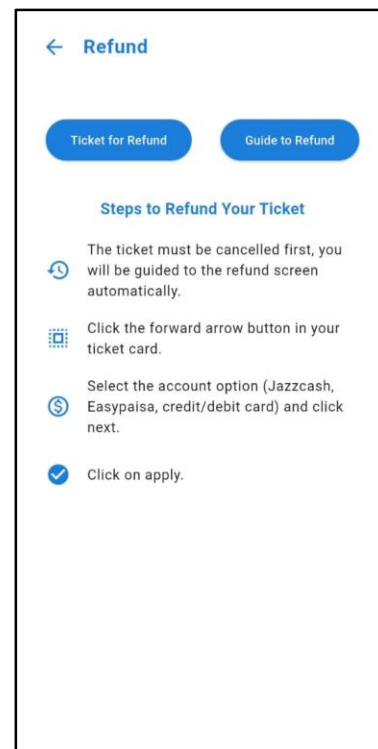


Figure 5.26: Refund Process UI

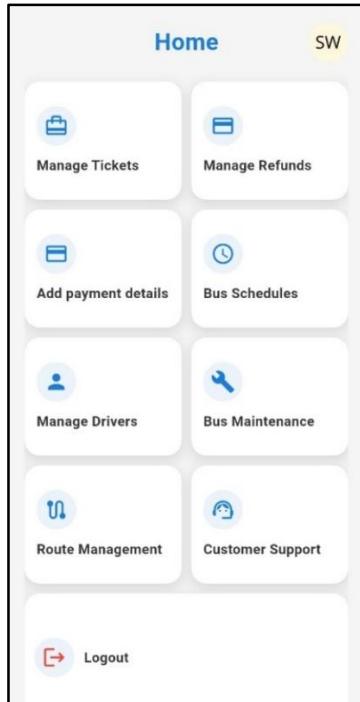


Figure 5.27: Homepage (Company Admin)

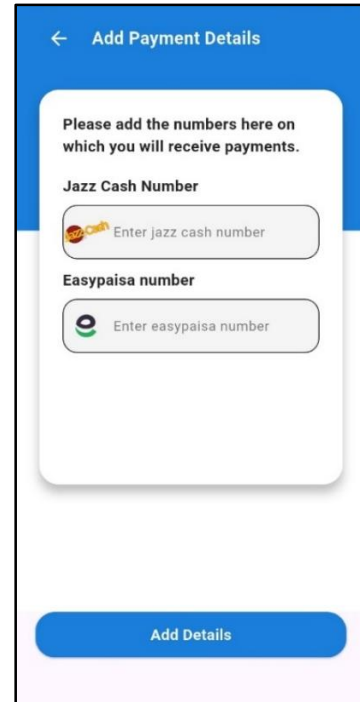


Figure 5.28: Payment Details UI

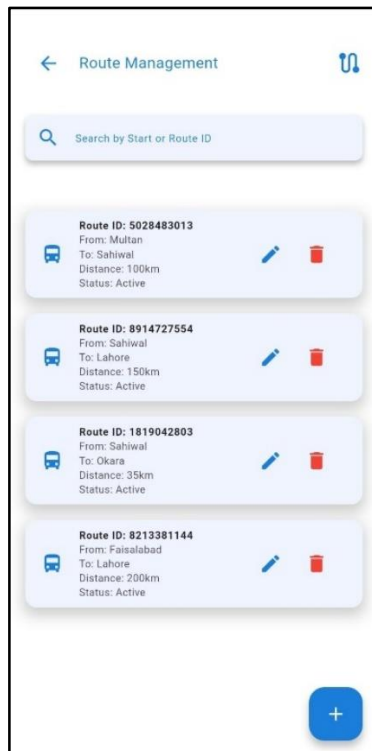


Figure 5.29: Route Management UI

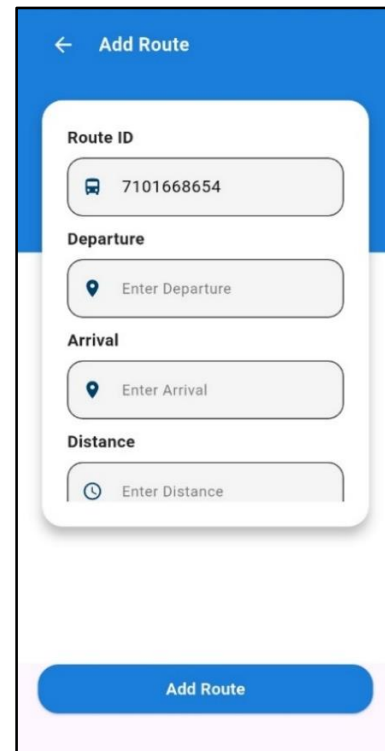


Figure 5.30: Add Route UI

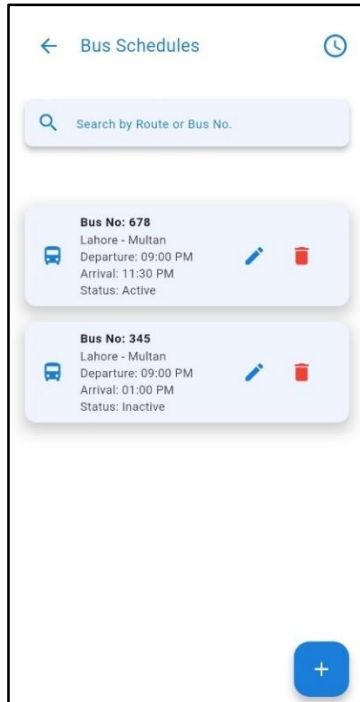


Figure 5.31: Bus Schedule UI

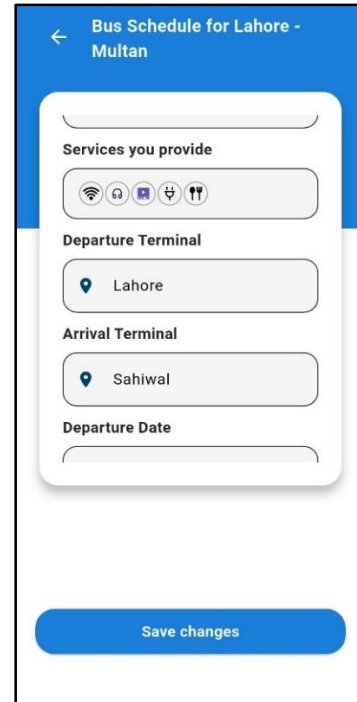


Figure 5.32: Add Schedule UI

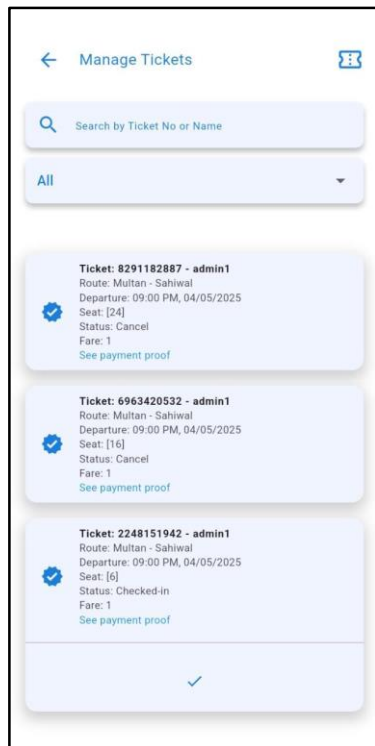


Figure 5.33: Manage Tickets UI

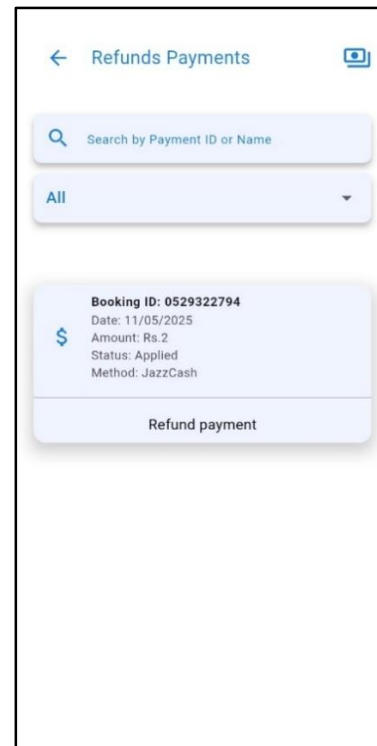
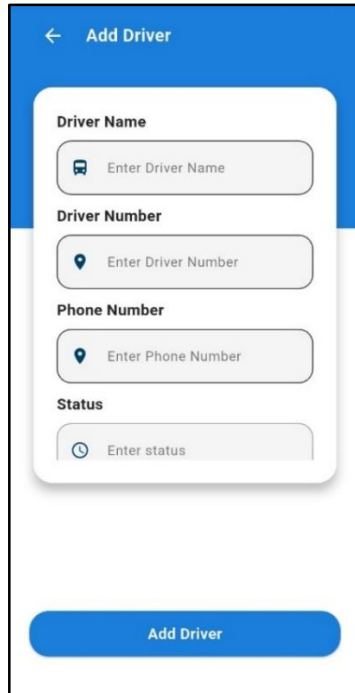


Figure 5.34: Refund Payment UI



Add Driver

Driver Name
Enter Driver Name

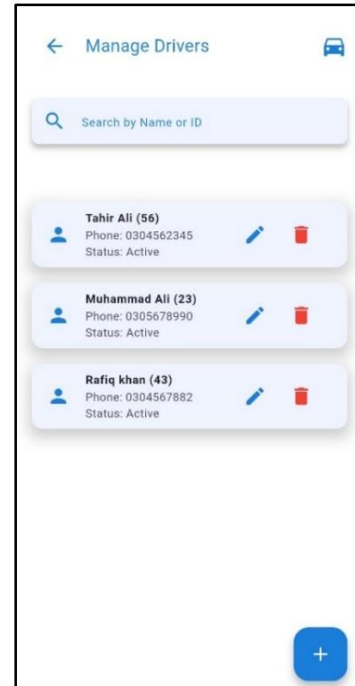
Driver Number
Enter Driver Number

Phone Number
Enter Phone Number

Status
Enter status

Add Driver

Figure 5.35: Add Driver UI



Manage Drivers

Search by Name or ID

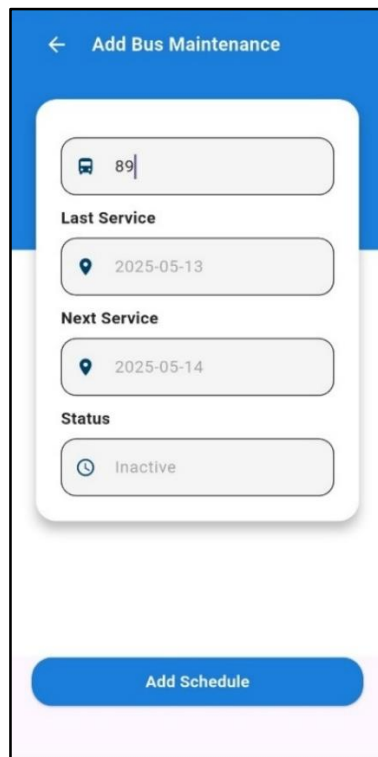
Tahir Ali (56)
Phone: 0304562345
Status: Active

Muhammad Ali (23)
Phone: 0305678990
Status: Active

Rafiq khan (43)
Phone: 0304567882
Status: Active

+

Figure 5.36: Manage Drivers UI



Add Bus Maintenance

89

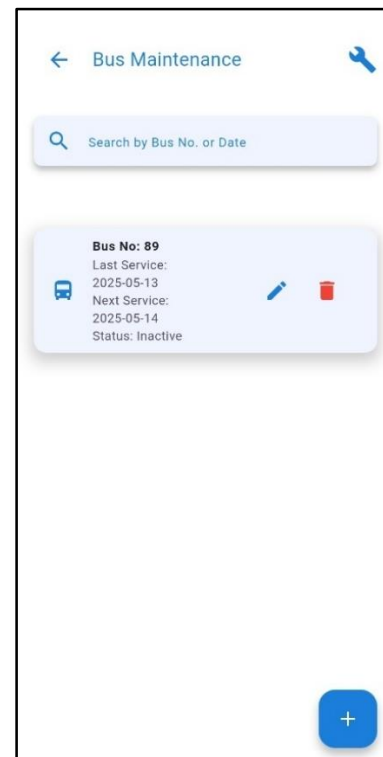
Last Service
2025-05-13

Next Service
2025-05-14

Status
Inactive

Add Schedule

Figure 5.37: Add Bus Maintenance UI



Bus Maintenance

Search by Bus No. or Date

Bus No: 89
Last Service: 2025-05-13
Next Service: 2025-05-14
Status: Inactive

+

Figure 5.38: Manage Bus Maintenance UI

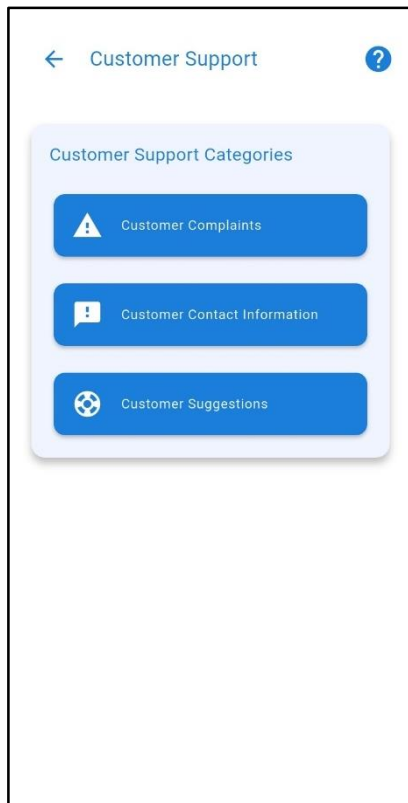


Figure 5.39: Company Admin Customer Support UI

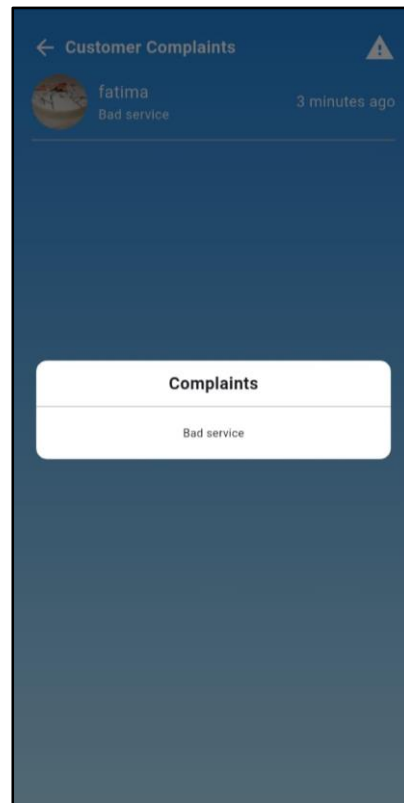


Figure 5.40: Company Admin Complain & Suggestions UI

Chapter 6

Testing and Evaluation

6. Testing and Evaluation

We performed unit and functional on the implemented modules as defined below,

6.1 Unit Testing

Following are the unit test cases of Safar's implemented functionalities,

UT-01: Registration

No	Test case/ Test Script	Attribute and Value	Expected Result	Result
1	Valid signup credentials	Name: Fatima Email: fatima@gmail.com , Password: 12345678, Phone number: 03038107655	Successful signup, token generated and redirected to the home page	Pass
2	Incomplete Password	Name: Fatima Email: fatima@gmail.com Password: 1234 Phone number: 03038107655	Error message: "Password must contain 8 characters."	Pass
3	Empty Password field	Name: Fatima Email: fatima@gmail.com Password: Phone number: 03038107655	Error message: "Password is required,	Pass
4	Empty Email field	Name: Fatima Email: Password: 12345678	Error message: "Email is required."	Pass

		Phone number: 03038107655		
5	Incorrect Email	Name: Email: fatima@ Password: 12345678 Phone number: 03038107655	Error message: “Email is invalid.”	Pass
6	Empty Name field	Name: Email: fatima@gmail.com Password: 12345678 Phone number: 03038107655	Error message: “Name is required.”	Pass

UT-02: Login

No	Test case/ Test Script	Attribute and Value	Expected Result	Result
1	Valid login credentials	Email: fatima@gmail.com , Password: 12345678	Successful signup, token generated and redirected to the home page	Pass
2	Empty Email field	Email: fatima@gmail.com , Password: 12345678	Error message: “Email is required.”	Pass
3	Incorrect Email	Email: fatima@ , Password: 12345678	Error message: “Email is invalid.”	Pass

4	Incomplete Password	Email: fatima@gmail.com , Password: 1234	Error message: “Password must contain 8 characters.”	Pass
5	Incorrect Password	Email: fatima@gmail.com , Password: 34527890	Error message: “Incorrect credential, Please try again.”	Pass
6	Empty Password Field	Email: fatima@gmail.com , Password:	Error message: “Password is required.	Pass

UT-03: Forgot Password

No	Test case/ Test Script	Attribute and Value	Expected Result	Result
1	Valid credential	Email: fatima@gmail.com	Message: “Email sent to your email address.”	Pass
2	Invalid email	Email: fatima12@gmail.com	Error message: “No account associated with this email.”	Pass
3	Incorrect Email	Email: fatima@ , Password: 12345678	Error message: “Email is invalid.”	Pass

UT-04: Add Payment Details

No	Test case/ Test Script	Attribute and Value	Expected Result	Result
1	Valid Credentials	Jazzcash Account: 03036785445	Message: “Numbers added successfully.”	Pass

		Easypesa Account: 03036785445		
2	Incorrect Email	Email: fatima@ , Password: 12345678	Error message: “Email is invalid.”	Pass

UT-05: Add Bus Schedules

No	Test case/ Test Script	Attribute and Value	Expected Result	Result
1	Valid Credentials	Bus Number: 234 Class Type: Business Prices: 2000 BusinessClass Price: 3000 No of seats: 35 Services you provide: Wifi, headphone etc Departure Terminal: Lahore Arrival Terminal: Multan Departure Date: 14 May, 2025 Arrival Date: 14 May2025 Departure Time: 1:00 AM Arrival Time: 5:00 AM Status: Active	Schedule is added.	Pass

2	Missing credentials	Bus Number: 234 Class Type: Business Prices: 2000 BusinessClass Price: 3000 No of seats: 35 Services you provide: Departure Terminal: Arrival Terminal: Departure Date: Arrival Date: Departure Time: Arrival Time:	Error message: “All fields are required.”	Pass
---	---------------------	--	--	------

UT-06: Add Drivers

No	Test case/ Test Script	Attribute and Value	Expected Result	Result
1	Valid Credentials	Enter Driver Name: Tahir Ali Driver Number: 43 Phone number: 03036784553 Status: Active	Driver is added.	Pass
2	Missing credentials	Enter Driver Name: Tahir Ali Driver Number: Phone number: 03036784553 Status: Active	Error message: “All fields are required.”	Pass

UT-07: Add Bus Maintenance

No	Test case/ Test Script	Attribute and Value	Expected Result	Result
1	Valid Credentials	Bus Number: 567 Last Service: 13/05/25 Next Service: 15/05/25 Status: Active	Bus added for Maintenance	Pass
2	Missing credentials	Bus Number: 567 Last Service: 13/05/25 Next Service: Status: Active	Error message: “All fields are required.”	Pass

UT-08: Add Routes

No	Test case/ Test Script	Attribute and Value	Expected Result	Result
1	Valid Credentials	Route ID: 75912 79991 Departure: Lahore Arrival: Multan Distance: 200 km Status: Active	New Route Added	Pass
2	Missing credentials	Route ID: Departure: Lahore Arrival: Multan Distance: 200 km Status: Active	Error message: “All fields are required.”	Pass

UT-09: Ticket Booking (Bus Search)

No	Test case/ Test Script	Attribute and Value	Expected Result	Result
1	Valid Credentials	Select Cities, Select Departure Date	Bus company options and Filters displayed	Pass
2	Missing credentials	Departure Date not Selected	Error message: “Departure Date is required for search.”	Pass

UT-10: Ticket Booking (Company Options and Filters)

No	Test case/ Test Script	Attribute and Value	Expected Result	Result
1	Valid Credentials	Select bus company, apply filter	Navigates to ticket display screen.	Pass
2	Missing credentials	Bus company not selected or filter not applied	Error message: “Please select all options.”	Pass

UT-11: Ticket Booking (Confirm you seat)

No	Test case/ Test Script	Attribute and Value	Expected Result	Result
1	Valid Credentials	Select single or multiple seat and Gender	Navigates to seat selection screen.	Pass
2	Missing credentials	Seat selection type not specified or Gender not selected	Error message: “Please select both options.”	Pass

UT-12: Ticket Booking (Seat Selection screen)

No	Test case/ Test Script	Attribute and Value	Expected Result	Result
1	Valid Credentials	Select a/the seat/seats	Navigates to ticket details screen.	Pass
2	Missing credentials	No seat selected,	An un-selectable “Confirm” button.	Pass

UT-13: Payment

No	Test case/ Test Script	Attribute and Value	Expected Result	Result
1	Valid Credentials	Upload right payment receipt on correct company number	Message: “Your payment has been successfully uploaded.”	Pass
2	Invalid credentials	Wrong receipt (wrong payment, wrong date, already submitted receipt)	Messages: 1. Submit Ticket of current date. 2. The payment proof is already accepted by the company.	Pass

UT-14: Complaints and Suggestions

No	Test case/ Test Script	Attribute and Value	Expected Result	Result
1	Valid Credentials	Select Complaint/ Suggestion radio button, select company	Complaint or suggestion sent	Pass
2	Invalid credentials	Company not selected, no text in the complaints/suggestions box.	Error Message: “All fields are required.”	Pass

6.2. Functional Testing

Following are the functional tests of Safar’s implemented functionalities.

FT-1: Login Functionality (User/Admin/Company Admin)

No	Test Case/ Test Script	Attribute and Value	Expected Result	Result
1	Login with valid credentials	Enter correct email & password	Redirect to respective dashboard	Pass
2	Login with invalid credentials	Enter incorrect email/password	Show error message: "Invalid Credentials"	Pass
3	Login with empty fields	Leave email/password blank	Show error: "Please enter all required fields"	Pass
4	Login with Google	Click "Login with Google"	Authenticated and redirected to user dashboard	Pass

FT-2: Registration Functionality (User/Company)

No	Test Case/ Test Script	Attribute and Value	Expected Result	Pass/Fail
1	Register new user	Fill in name, email, password, etc.	Account created; user redirected to login page	Pass
2	Register company	Fill in company name, contact, etc.	Request sent to Main Admin for verification	Pass
3	Register with missing fields	Leave one or more fields empty	Show appropriate validation messages	Pass
4	Register with already used email	Use an email that is already registered	Show error: "Email already exists"	Pass

FT-3: Forgot Password

No	Test Case/ Test Script	Input/Action	Expected Result	Pass/Fail
1	Submit registered email	Enter valid email	Reset link sent via email	Pass
2	Submit unregistered email	Enter an email not in system	Show error: "Email not found"	Pass
3	Submit empty email	Leave field empty	Show error: "Email is required"	Pass

FT-4: Company Admin Panel

No	Test Case/ Test Script	Action	Expected Result	Pass/Fail
1	Add payment numbers	Add JazzCash/EasyPaisa numbers	Numbers saved successfully	Pass
2	Add/edit routes	Create a new bus route and schedule	Route visible on user side during search	Pass
3	Manage drivers/buses	Add/edit/delete driver/bus	Updates reflected in system	Pass
4	View user suggestions /complaints	Go to "Contact" section	Messages are listed	Pass
5	Process refund	Approve refund and upload screenshot	User notified, refund marked as completed	Pass

FT-5: Ticket Booking Flow (User Side)

No	Test Case/ Test Script	Input/Action	Expected Result	Pass/Fail
1	Select route	Choose departure, arrival city, and date	Filter popup appears	Pass
2	Filter companies	Select company, class (economy/business)	Matching tickets displayed	Pass
3	View ticket details	Click "View Details"	Show full ticket details	Pass
4	Select seat (gender-based)	Choose seat(s) and gender	Valid colors; warning if gender mismatch	Pass
5	Confirm ticket	Click "Confirm Booking"	Show payment screen	Pass

FT-6: Payment Method Screen

No	Test Case/ Test Script	Input/Action	Expected Result	Pass/Fail
1	Select JazzCash/Easy-Paisa	Tap one payment method	Number to send payment shown	Pass
2	Upload payment screenshot	Attach screenshot	Ticket marked as "Pending"	Pass
3	TID verification by company	Admin verifies transaction ID	Status changed to "Confirmed"	Pass

FT-7: Profile & Navigation

No	Test Case/ Test Script	Input/Action	Expected Result	Pass/Fail
1	View/edit profile	Change name, phone, etc.	Changes saved	Pass
2	Open drawer	Access navigation menu	All options visible: Suggestion, Chatbot, Logout, etc.	Pass
3	View ticket history	Navigate to "My Tickets"	Show all booked/cancelled tickets	Pass

6.3. Integration Testing

Following are the integration test cases of Fitness Pal's implemented functionalities:

IT-01: Login & Authentication Integration (Firebase)

No	Integration Components	Test Scenario	Expected Result	Status
1	Firebase Auth	Verify login for user with valid credentials	User/Admin dashboard is displayed	Pass
2	Firebase Auth + Google Sign-In	Verify login with Google account	Redirects to dashboard after Google auth	Pass
3	Firebase Auth + EmailJS	Verify password reset email is sent via EmailJS	Email sent with reset link	Pass

IT-02: Signup & Registration Integration (Firebase, Supabase)

No	Integration Components	Test Scenario	Expected Result	Status
1	Firebase Auth + Supabase	Verify new user registration and profile data storage	Account created and user profile saved in DB	Pass
2	Firebase Auth + Supabase	Register a company and verify main admin approval flow	Company status set as 'pending' until approved	Pass

It-03: Communication & Notification Integration (EmailJS)

No	Integration Components	Test Scenario	Expected Result	Status
1	Firebase + EmailJS	Reset password sends email	User receives reset email via EmailJS	Pass

2	Drawer UI + EmailJS	User submits complaint/suggestion	Email sent to support/admin	Pass
---	---------------------	-----------------------------------	-----------------------------	------

It-04: Company Admin Features Integration (Supabase)

No	Integration Components	Test Scenario	Expected Result	Status
1	Supabase DB	Company admin adds JazzCash and EasyPaisa numbers	Numbers saved in Supabase	Pass
2	Supabase DB	Admin adds route and schedule	Route saved and displayed on user side	Pass
3	Supabase + EmailJS	Admin sends refund and uploads proof	User gets email notification and status updated	Pass

IT-05: User Booking Flow Integration (Supabase)

No	Integration Components	Test Scenario	Expected Result	Status
1	Supabase	User selects cities and date	Available companies and filters displayed	Pass
2	Supabase	Tickets shown based on filters	Tickets listed correctly	Pass
3	Supabase	User selects a seat	Seat status updated based on gender logic	Pass

It-06: User Profile and Support Integration (Supabase, EmailJS)

No	Integration Components	Test Scenario	Expected Result	Status
1	Supabase	View and update profile info	Info updated in Supabase	Pass
2	Supabase + EmailJS	User applies for refund and uploads proof	Admin notified and refund status updated	Pass

.

Chapter 7

Conclusion and Future Work

7. Conclusion and Future Work

This chapter summarizes the project's achievements and contributions. It also highlights areas for potential improvement and expansion in future versions of the project.

7.1 Conclusion

The **SAFAR app** is a comprehensive and user-friendly bus travel platform that simplifies and digitizes the entire bus ticketing and travel experience. It connects users, transport companies, and administrative bodies in a centralized ecosystem. The app effectively handles:

1. User account management (Login, Signup, Google Sign-In, Forgot Password).
2. Company registration and verification (Main Admin Approval).
3. Bus and route management by companies (including drivers, schedules, payment numbers).
4. Seat selection with a gender-sensitive system (visual differentiation and alerts).
5. Ticket booking with payment via JazzCash and EasyPaisa.
6. Ticket verification and refund mechanism.
7. Admin dashboard for company and main admin to manage operations.
8. User support through suggestions, complaints, and chatbot integration.

It ensures a secure, organized, and intuitive system for booking intercity bus travel, reducing manual overhead, and increasing convenience for travelers and transport service providers.

7.2 Future Work

Here are several potential improvements and features that can be added to enhance the app:

1. Real-Time Bus Tracking

Integrate GPS for **live location tracking** of buses. Allow users to check estimated arrival times and track the journey in real time [13].

2. Online Payment Gateway Integration

Add **official in-app payment gateways** (like Stripe, PayFast, or direct bank transfers) to make transactions smoother and more automated. Auto-verification instead of manual screenshot upload.

3. E-Ticket and QR Code Generation

Generate QR-coded e-tickets after successful booking. Let conductors scan tickets using the SAFAR admin/conductor app for verification.

4. In-App Notifications

Use Firebase Cloud Messaging (FCM) for **push notifications** for booking confirmations, delays, cancellations, etc.

5. Multilingual Support

Offer the app in **multiple languages** like Urdu, Punjabi, or regional languages for better accessibility [14].

6. Reviews & Ratings

Allow users to **rate bus companies**, specific trips, or drivers. Help future travelers make informed choices based on feedback.

References

- [1] indeed, "indeed," indeed, 4 3 2025. [Online]. Available: <https://www.indeed.com/career-advice/career-development/how-to-write-a-brief>. [Accessed 14 5 2025].
- [2] Techtarget, "techtarg-OOP," techtarget, 6 2024. [Online]. Available: <https://www.techtarget.com/searchapparchitecture/definition/object-oriented-programming-OOP>. [Accessed 14 5 2025].
- [3] IBM, "IBM, Mobile Application Development," IBM, [Online]. Available: <https://www.ibm.com/think/topics/mobile-application-development>. [Accessed 14 05 2025].
- [4] V. Nikitin, "itransition/Software Design Methodology," itransition, 16 4 2024. [Online]. Available: <http://itransition.com/software-development/methodologies>. [Accessed 14 5 2025].
- [5] V. Kanade, "spiceworks, what is human computer interaction?," spiceworks, 7 7 2022. [Online]. Available: <https://www.spiceworks.com/tech/artificial-intelligence/articles/what-is-hci/>. [Accessed 14 5 2025].
- [6] S. Laoyan, "asana, what are agile methodologies," asana, 20 2 2025. [Online]. Available: <https://asana.com/resources/agile-methodology>. [Accessed 14 5 2025].
- [7] Requirements, "Requirements, functional requirements," Requirements, 12 7 2024. [Online]. Available: <https://requirements.com/Content/What-is/what-are-functional-requirements>. [Accessed 14 5 2025].
- [8] G. Kruger, "perforce, non-functional requirements," perforce, 8 8 2024. [Online]. Available: <https://www.perforce.com/blog/alm/what-are-non-functional-requirements-examples>. [Accessed 14 5 2025].
- [9] fortinet, "fortinet, what is mobile app security?," fortinet, 2025. [Online]. Available: <https://www.fortinet.com/resources/cyberglossary/mobile-app-security>. [Accessed 14 5 2025].
- [10] T. T. Contributor, "Tech Target, scalability," Tech Target, 7 2024. [Online]. Available: <https://www.techtarget.com/searchdatacenter/definition/scalability>. [Accessed 14 5 2025].
- [11] Derrell, "inmotionhosting," inmotionhosting, 10 5 2024. [Online]. Available: <https://www.inmotionhosting.com/support/edu/laravel/model-view-controller-intro/>. [Accessed 14 5 2025].

- [12] G. f. Geeks, "Geeks for Geeks, UML diagrams," Geeks for Geeks, 3 1 2025. [Online]. Available: <https://www.geeksforgeeks.org/unified-modeling-language-uml-activity-diagrams/>. [Accessed 14 5 2025].
- [13] K. M, "fleetroot, real time bus tracking," fleetroot, 17 2 2022. [Online]. Available: <https://www.fleetroot.com/blog/how-does-real-time-bus-tracking-work/>. [Accessed 14 5 2025].
- [14] G. Harper, "TopDevelopers, Multilingual App Development," TopDevelopers, 28 8 2023. [Online]. Available: <https://www.topdevelopers.co/blog/multilingual-app-development-casting-light-factors/>. [Accessed 14 5 2025].